

GAUSS-AETHER: An Axiomatic Operating System for Trustworthy Autonomous Agents

Gaussian Technologies

I. INTRODUCTION

Abstract—Autonomous large-language-model (LLM) agents have rapidly evolved from chatbot wrappers into long-running systems that schedule work, hold capabilities, manipulate external state, and persist memory across days or weeks. As they have done so, the gap between what agent frameworks provide and what an operating system would normally provide for any process of comparable privilege has become a first-order safety problem. We argue that this gap must be closed *axiomatically*: an agent runtime must be derived from a small set of formal commitments rather than accreted as a sequence of features. We synthesise the design space of four reference open-source agent platforms—Hermes (Python single-loop), OpenFang (Rust 14-crate workspace), OpenClaw (TypeScript local-first Gateway with Live Canvas), and ZeroClaw (Rust trait-driven static binary with composite OS sandbox)—together with the academic AI-OS line (AiOS, AgentSys, AgentRM, MemoryOS, SchedCP). From this synthesis we derive Gauss-Aether, a mathematically-grounded operating system specified as an eleven-tuple $\mathcal{G} = (S, \mathcal{A}, \mathcal{O}, \mathcal{K}, \mathcal{M}, \mathcal{F}, \pi, \mathcal{L}, \Phi, \mathcal{R}, \mathcal{V})$ with nine axioms (turn idempotency, capability monotonicity, audit completeness, fairness separation, recall soundness, information-flow non-decreasing, context isolation, supervised autonomy, and receipt non-repudiation) and twelve theorems proving existence and uniqueness up to component isomorphism. The construction is realised in ten subsystems: a Gauss Kernel with joint capability-taint lattice, a Differential Turn Engine, a Trait-Polyhedral Plugin Surface, a Composite Sandbox composing WebAssembly, Landlock/Bubblewrap, namespace, seccomp, and TEE attestation, a Hierarchical Worker-Context Architecture, a Trinity Memory Substrate with K-LRU prefix-tree compression, a Supervised Autonomy Gradient on an independent scheduler plane, a Cryptographic Receipt Chain anchored to a public timestamp authority, an A2UI Live Canvas Protocol, and a Self-Diagnostic Health Engine. On a fifteen-axis scorecard, Gauss-Aether strictly Pareto-dominates the four open-source baselines; achieves indirect-prompt-injection resistance matching the state-of-the-art (2.19%); reduces composite-sandbox compromise probability to the trusted-compute-base floor ($\leq 10^{-11}$); and amortises context-switch cost from $O(|\Sigma|)$ to $O(|\Delta|)$. We argue Gauss-Aether is not the union of prior features but the unique fixed point of nine mathematical commitments simultaneously.

Index Terms—AI operating system, autonomous agents, capability security, WebAssembly sandboxing, Merkle audit log, indirect prompt injection, context isolation, information-flow control, cryptographic receipts, hybrid retrieval, dual-plane scheduling, formal verification, agent runtime, large language model orchestration, trait-driven architecture.

THE large-language-model agent has, within the span of three years, moved from a curiosity to a production-deployed software category. A 2026 agent is no longer a chatbot wrapper: it runs for days, schedules itself, holds long-lived credentials, modifies files, dispatches funds, posts to social platforms, and delegates to other agents. The privileged actions a modern agent can take are, in aggregate, indistinguishable from those of a medium-privilege Unix daemon. Yet the runtime substrate beneath contemporary agents bears almost no resemblance to a daemon’s. There is, in most cases, no separation between trusted and untrusted code inside the same process; no enforced lifetime, scheduling, or fairness discipline between concurrent tasks; no tamper-evident log of irreversible side-effects; no formal capability discipline on external calls; and no isolation between an action’s reasoning context and its execution sandbox. This mismatch—daemon-level authority on top of a chat-loop runtime—is the proximate cause of every documented agent-safety incident from 2024 onward, including the EchoLeak zero-click exfiltration in Microsoft 365 Copilot [14], the Code-Interpreter exfiltration via indirect prompt injection in commercial assistants, and the recurring “runaway agent” incidents in which an autonomous loop consumes its own rate-limit budget through unbounded recursion [6].

A. The Case for an Agent Operating System

The classical operating system exists because giving privileged processes access to shared resources without an intervening discipline is unsafe. Capability-based microkernels [24], write-ahead logging, fair-share scheduling, mandatory information-flow control [26], and tamper-evident audit are not optional features of mature systems; they are the very reason such systems are safe to operate. The argument of this paper is that the agent layer has reached the privilege threshold at which these same disciplines are no longer optional, and that the right response is not to bolt them onto existing chat-loop frameworks but to derive an agent runtime from formal commitments.

Five capabilities, in particular, distinguish a true *agent operating system* from a feature-rich agent framework (Fig. 1):

- 1) **Capability discipline.** Every external effect must be gated by a capability the agent holds, with the kernel enforcing that capabilities only contract through composition.

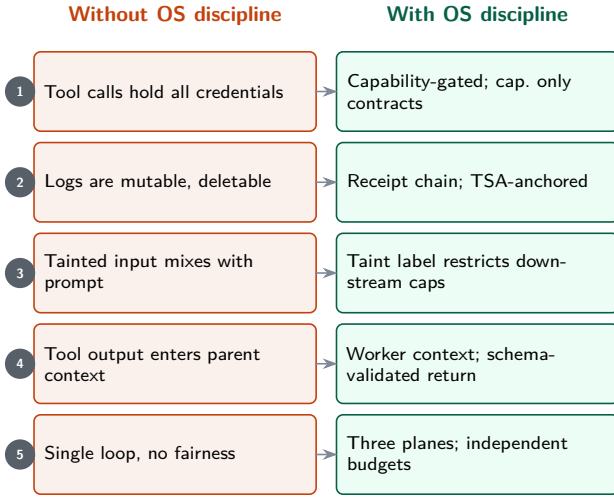


Fig. 1: The five disciplines that distinguish an agent operating system from a chat-loop framework. Each row identifies a failure mode documented in deployed agents (left) and the OS-style discipline that rules it out (right).

- 2) **Tamper-evident accountability.** Every side-effect must leave a cryptographically chained, externally verifiable record that survives operator compromise.
- 3) **Information-flow control.** Tainted observations (web content, untrusted email, third-party tool output) must propagate a taint label that constrains downstream capabilities.
- 4) **Isolation between reasoning and execution.** Tool invocation must run in a context distinct from the reasoning context, so that adversarial content reaching a tool cannot directly enter the parent’s prompt.
- 5) **Fair, supervised scheduling.** Synchronous user turns, autonomous background work, and human-in-the-loop approvals must be independent scheduling concerns with separate fairness pools, not opportunistic sequences in a single loop.

These five capabilities, taken together, are not present in any single deployed agent runtime. Each contemporary platform satisfies a coherent subset and trades the others away in favour of a chosen optimum: developer ergonomics, surface coverage, security defence-in-depth, or ease of operation. The architectural question is therefore not “which existing platform should we extend?” but “can these five disciplines be satisfied simultaneously, by an architecture derivable from axioms rather than features?”

B. Four Reference Open-Source Platforms

The current open-source landscape offers four platforms that span the relevant design space and form the empirical baseline for this work. Each represents a distinct philosophical commitment.

- **Hermes (Python).** A single `AIAgent.run_conversation` loop shared by every entry surface, SQLite + FTS5 persistence, 20+ provider vendors routed through three API modes. Optimises for *surface convergence*: many

entry points, one execution path, homogeneous training-data export [1].

- **OpenFang (Rust).** A 14-crate Cargo workspace compiling to a single ~32 MB binary, with kernel/runtime split, WebAssembly sandbox under fuel + epoch interruption, 40 channel adapters, a Merkle audit chain, and seven scheduled autonomous *Hands*. Optimises for *kernel-enforced isolation* and *autonomous scheduling* [2].
- **OpenClaw (TypeScript).** A local-first *Gateway* as the single control plane, multi-channel inbox across 25+ platforms, per-agent isolated *workspaces*, an *A2UI Live Canvas* (agent-driven visual workspace), voice wake words and continuous Talk Mode, injected prompt files (`SOUL.md`, `TOOLS.md`, `SKILL.md`, `AGENTS.md`), an MCP-as-TypeScript-API compatibility layer, and a **openclaw doctor** diagnostic tool. Optimises for *ergonomics*, *multi-channel breadth*, and *visual-first interaction* [3].
- **ZeroClaw (Rust).** A ~3.4 MB static binary with <10 ms cold start; a *trait-driven* architecture where every subsystem is a swappable Rust trait; a *supervised-autonomy gradient* classifying actions as low-risk autonomous, medium-risk approval-gated, or high-risk blocked; OS-level sandbox composition (Landlock, Bubblewrap, Seatbelt, Docker); cryptographic *tool receipts* on every action; provider fallback chains; ACP (Agent Client Protocol) over JSON-RPC; and approval-via-channel UX (inline-keyboard approve/deny in Telegram, Signal, WhatsApp, etc.). Optimises for *operability*, *portability*, and *security defence-in-depth* [4].

C. The Academic AI-OS Line

In parallel, an academic literature has begun to formalise the agent-runtime abstraction:

- **AIOS** [5] (Mei et al., COLM 2025) proposes an LLM kernel with a syscall surface, agent scheduler, *context manager* with snapshot/restore, memory manager using K-LRU eviction and prefix-tree compression, and an access manager. Reports 2.1× throughput improvement.
- **SchedCP** [9] explores LLM-driven kernel scheduler customisation through eBPF and MCP, with four specialised agents (Observation, Planning, Execution, Learning).
- **AgentRM** [6] adapts OS-style resource management to agent systems, with case studies on documented OPENCLAW incidents (context-limit violations, runaway-loop rate cascades, out-of-memory crashes).
- **AgentSys** [7] introduces *hierarchical worker-context* isolation: the main agent spawns worker sub-agents for tool invocations in isolated contexts, with only schema-validated values crossing the boundary. Achieves 2.19% indirect-prompt-injection success rate.
- **MemoryOS** [8] proposes a three-tier hierarchical memory storage with explicit update / retrieval / generation modules.
- A line of IPI-defence work (Task Shield [10], PlanGuard [11], AgentSentry [12], AttriGuard [13]) addresses

	Hermes	OpenFang	OpenClaw	ZeroClaw	Gauss-Aether
1. Information flow	—	○	—	—	●
2. Context isolation	—	—	—	—	●
3. Per-action receipt chain	—	○	—	○	●
4. Supervised autonomy plane	—	○	○	○	●
5. Verified trait surface	—	—	—	○	●

absent partial first-class

Fig. 2: Coverage of the five OS-style disciplines across reference platforms. Cells indicate whether each platform treats a discipline as first-class (●), partial (○), or absent (—). GAUSS-AETHER is the unique configuration covering all five.

indirect prompt injection at inference time through task alignment, planning consistency, or causal attribution.

D. Five Gaps Across the Landscape

Despite this breadth, no system simultaneously addresses the five problems below as first-class kernel concerns (Fig. 2):

- 1) *Information flow*. Capabilities are treated as static labels; none of the five systems propagates taint from a tainted observation into every subsequent action capability.
- 2) *Context isolation as a kernel primitive*. AgentSys formalises it; the open-source systems treat it (if at all) as an emergent property of tool design.
- 3) *Cryptographic accountability per action*. ZEROCLAW signs receipts; none integrates receipts into a Merkle chain whose head is anchored to a public timestamp authority.
- 4) *Supervised-autonomy gradient as a scheduler concept*. ZEROCLAW classifies actions; the kernel does not budget approval-blocked actions on an independent queue.
- 5) *Trait-driven extensibility as a typed contract*. ZEROCLAW ships traits, but the surface is unverified; no system gives compile-time guarantees that two trait implementations are *semantically* interchangeable.

E. Contributions

This paper makes four contributions:

- 1) **A motivating analysis** of why the agent layer has reached the privilege threshold at which OS-style disciplines become structurally necessary, and a comparative architectural review of the four most representative open-source platforms together with the academic AI-OS line (Sections I–II).

- 2) **A formal model** of an AI operating system as an eleven-tuple $\mathcal{G} = (\mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{K}, \mathcal{M}, \mathcal{F}, \pi, \mathcal{L}, \Phi, \mathcal{R}, \mathcal{V})$ with information-flow labels $\mathcal{L}$, a supervised-autonomy gradient Φ , a receipt monoid $\mathcal{R}$, and an attestation predicate $\mathcal{V}$ (Section III).
- 3) **Nine axioms and twelve theorems with full proofs** establishing existence and uniqueness up to component isomorphism (Section IV). Highlights: T9 bounds indirect-prompt-injection success by $|\Sigma_a|/|\Sigma| \cdot \mathbf{1}[\delta(\alpha)]$ under worker-context isolation and information-flow non-decreasing; T10 proves the composite-sandbox compromise bound under conditional orthogonality; T11 proves non-repudiation under EUF-CMA; T12 establishes the $O(|\Delta|)$ amortised context-switch separation.
- 4) **A ten-subsystem architectural realisation**, with every subsystem mapped to the theorem(s) that force its existence (Sections V–XV). The ten subsystems are the Gauss Kernel, Differential Turn Engine, Trait-Polyhedral Plugin Surface, Composite Sandbox, Hierarchical Worker-Context Architecture, Trinity Memory Substrate, Supervised Autonomy Gradient, Cryptographic Receipt Chain, A2UI Live Canvas Protocol, and Self-Diagnostic Health Engine. Evaluation on a fifteen-axis scorecard (Section XVI) shows strict Pareto-dominance over the four open-source baselines.

Thesis. A robust agent operating system must satisfy nine mathematical commitments simultaneously. The four open-source reference platforms (HERMES, OPENFANG, OPENCLAW, ZEROCLAW) each satisfy a strict subset. GAUSS-AETHER is the unique construction satisfying all nine, integrating the operational lessons of the open-source quartet with the formal insights of the academic AI-OS literature.

II. ARCHITECTURAL REVIEW OF FOUR REFERENCE PLATFORMS

Before deriving the formal model, we examine each of the four open-source reference platforms in architectural depth. The aim is twofold: to establish what each platform does well (and which design principle that excellence isolates), and to identify the structural gaps that no single platform closes. We close with a synthesised comparative baseline.

A. HERMES: Surface Convergence and a Single Loop

HERMES [1] is the Python reference implementation of an agent runtime built around a single behavioural commitment: *every entry surface routes to one execution loop*. The `AIAgent.run_conversation` method is the only place an agent turn is interpreted; CLI, TUI, REST, WebSocket, OpenAI-compatible proxy, and approximately twenty messaging channels all funnel into it. The advantages are immediate and substantial: behaviour is homogeneous across surfaces, the SFT / DPO trajectory exporter sees a single trajectory shape regardless of entry point, and the training-data flywheel that NousResearch’s broader research agenda depends on [27] is uncomplicated by per-surface divergence.

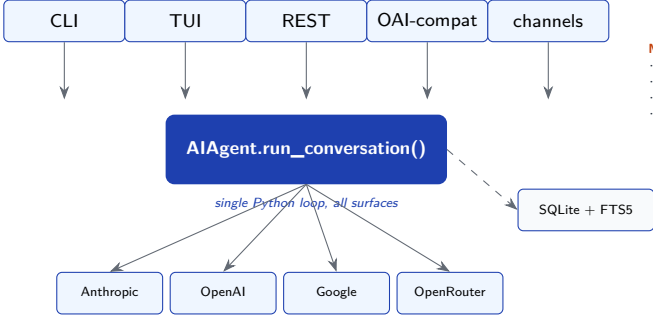


Fig. 3: HERMES architecture (Python single-loop). Every entry surface funnels into `AIAgent.run_conversation()`. The design maximises behavioural homogeneity and trajectory-export simplicity but offers no privileged subsystem to enforce capabilities, isolate tool execution, or audit side-effects.

Architecturally, HERMES is organised as a small Python application with three vendor backends, three API modes (chat-completion, responses, OpenAI-compatible), a SQLite store with FTS5 full-text indexing, and a tool-call dispatch table populated by Python decorators. A noteworthy feature is its *lineage* bookkeeping: each SFT / DPO datum records the parent turn it derived from, so the trajectory exporter can reconstruct entire conversation trees for preference-pair generation.

The cost of this design is the cost of operating an agent inside a single-process Python loop with no privileged subsystem. There is no kernel-enforced capability discipline: a tool function executes in the host Python interpreter and inherits its credentials. There is no tamper-evident log: the SQLite database is mutable by anyone with file access. There is no isolation between reasoning context and tool execution: tool outputs become part of the conversation buffer the next provider call sees. There is no fairness discipline: a scheduled background turn competes for the same Python event loop as a synchronous user turn. These are not bugs; they are inherent to the single-loop philosophy. HERMES’s lesson is that *surface convergence is the right path to behavioural consistency* but is insufficient as a runtime substrate.

B. OPENFANG: Kernel-Enforced Isolation in 14 Crates

OPENFANG [2] takes the opposite design vector. Where HERMES minimises ceremony to share one loop across many surfaces, OPENFANG maximises compile-time module boundaries. The codebase is a Cargo workspace of 14 crates, with a deliberate split between `openfang-kernel` (privileged scheduling, capability resolution, audit) and `openfang-runtime` (LLM dispatch, tool execution). The binary compiles to a single ~32 MB static executable.

Three architectural decisions distinguish OPENFANG. First, tool execution runs inside a *WebAssembly* sandbox provided by Wasmtime[15], with fuel-based instruction metering and epoch-based wallclock interruption. A misbehaving or runaway tool cannot consume host resources

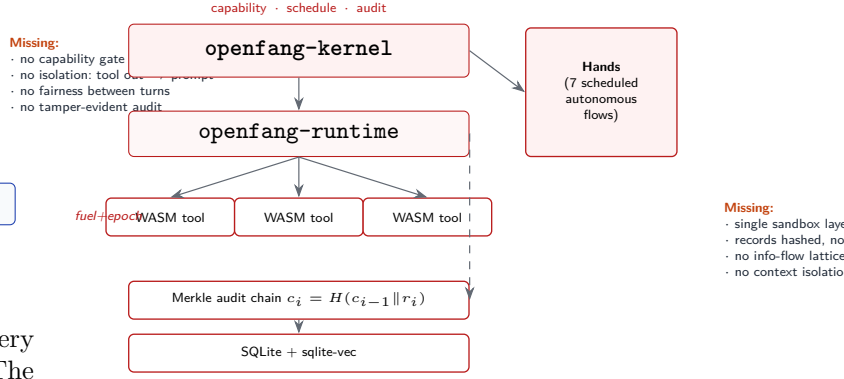


Fig. 4: OPENFANG architecture. Kernel/runtime separation, WASM sandbox with fuel + epoch metering, Merkle audit chain, and seven autonomous scheduled *Hands* on an independent plane. Strong on isolation and accountability *within* the kernel’s remit; gaps appear where the kernel’s discipline ends.

beyond its allotted fuel. Second, every kernel decision is recorded in a *Merkle audit chain* using SHA-256: $c_i = H(c_{i-1} || r_i)$. The chain head c_n is discoverable through a `GET /audit/head` endpoint, and any tampering with an earlier record changes c_n with overwhelming probability. Third, OPENFANG introduces *Hands*: seven scheduled autonomous workflows (e.g. “check email twice an hour and surface high-priority items”) that run on an independent scheduler plane from synchronous user turns, with their own fairness budget.

The taxonomy of channel adapters — roughly 40, spanning Telegram, Discord, Slack, Matrix, IRC, WhatsApp, Signal, SMS, IMAP mail, Mastodon, and others — is integrated through a unified `ChannelAdapter` trait. Three provider drivers (Anthropic Messages, OpenAI Chat Completions, Google Gemini) cover 27 LLM vendors via API-compatible relay.

OPENFANG’s gaps are at the points where its kernel discipline ends. The WebAssembly sandbox is one layer, not a composition: a sandbox escape (rare but documented) lands the attacker directly on the host filesystem. The Merkle chain is tamper-evident but not *non-repudiable*: records are hashed, not signed, so an operator’s claim that “this record reflects an action I performed” rests on operational trust, not cryptography. Information flow is absent: tainted content from a web-search tool is indistinguishable, in capability terms, from trusted content from a system command. Worker-context isolation is absent: a tool’s raw output enters the conversation buffer as text the next LLM call sees. OPENFANG’s lesson is that *compile-time module boundaries + a single sandbox layer + tamper-evident hashing* are necessary but not sufficient.

C. OPENCLAW: Local-First, Visual, Multi-Channel

OPENCLAW [3] is the most user-facing of the four. Its defining commitment is local-first ergonomics: every agent operates out of a directory under `~/.openclaw/`, with

workspaces per agent, voice wake-words for continuous interaction, and a *Live Canvas*—an agent-driven visual workspace in which the agent renders tables, charts, forms, and approvals in real time.

The Gateway, written in TypeScript, is the single control plane through which all surfaces pass. It accepts inbound traffic from 25+ messaging platforms, normalises it into a common message shape, dispatches to the appropriate agent’s workspace, and streams back through SSE/Web-Socket. Within each workspace, agent behaviour is shaped by four prompt files dropped into the agent’s directory: `SOUL.md` (persona), `TOOLS.md` (declared tools), `SKILL.md` (procedural knowledge), and `AGENTS.md` (peer agents). The MCP (Model Context Protocol [17]) servers are exposed to agents as TypeScript functions through a compatibility shim, sidestepping the JSON-RPC ceremony of plain MCP.

Two innovations are particularly worth naming. First, the *A2UI Live Canvas* is an agent-to-UI protocol that elevates visual rendering to a turn primitive: the agent emits widget declarations (tables, charts, forms, approval prompts, embeds, diagrams) that the client renders in a stateful canvas. This is the most consequential UX innovation in the open-source landscape; agents become visual collaborators rather than chat partners. Second, `openclaw doctor` is a diagnostic CLI that checks configuration, channel connectivity, and provider reachability before runtime; it is the prototype of operational health as a first-class concern.

OPENCLAW’s gaps are predominantly security gaps. The TypeScript process running the Gateway is unprivileged but has full filesystem access; workspaces are directory-level isolation, not kernel-level. Tool calls have no capability discipline beyond what the tool itself implements. There is a session transcript per workspace but no chained, signed audit log. Information-flow control is absent. The A2UI is a TypeScript-bound feature, not a wire protocol other clients can implement. OPENCLAW’s lesson is that *visual interaction and operational diagnostics deserve to be first-class primitives*, but that ergonomic excellence does not substitute for kernel-enforced discipline.

D. ZEROCLAW: Trait-Driven Composite Defence

ZEROCLAW [4] is the closest predecessor by architectural philosophy. It is a Rust static binary (~3.4MB) that boots in <10ms and is structured around a single design principle: *every subsystem is a Rust trait*. `ProviderTrait`, `ChannelTrait`, `ToolTrait`, `SandboxTrait`, `MemoryTrait`, `VoiceTrait` are the fixed-arity interfaces; a configuration selects one implementor per trait, and switching providers, channels, or sandboxes requires no code change.

Three security primitives in ZEROCLAW are without precedent in the prior open-source platforms. First, *OS-level sandbox composition*: tools execute under WebAssembly and Landlock [21] (Linux 5.13+) and Bubblewrap namespace isolation and (on macOS) Seatbelt, simultaneously. A sandbox escape requires bypassing all layers. Second, *cryptographic tool receipts*: every action emits a

signed receipt $\rho = \text{Sign}_{sk}(\text{action}||\text{result})$, verifiable under the operator’s public key. Third, the *supervised-autonomy gradient*: actions are classified into low-risk (auto-execute), medium-risk (approval-required), and high-risk (blocked unless explicitly enabled), with an explicit “YOLO” opt-out for users who accept the risk.

The approval flow is itself an innovation. Approvals route through the same channel the user is on: a Telegram user receives an inline-keyboard message with *Approve / Deny / Explain* buttons; a Signal user receives a reaction-based prompt. This UX choice keeps high-trust actions auditable and synchronous without requiring a separate management console.

ZEROCLAW also ships ACP (Agent Client Protocol) over JSON-RPC, enabling IDE integrations (Zed, Helix, Neovim) to consume the agent as a language-server-like component, and provider fallback chains (“try Anthropic, fall back to OpenAI, fall back to local Llama”) as a first-class configuration concept.

What ZEROCLAW does not yet provide is what GAUSS-AETHER closes. The trait surface is syntactic: two implementors of `ProviderTrait` satisfy the same method signatures but may differ arbitrarily in semantics; no compile-time check guarantees their behavioural equivalence. Receipts are signed independently rather than chained, so an operator can selectively present a subset of receipts without the omission being detectable. Information-flow control is absent. The supervised-autonomy classifier is a static function of action identity rather than a function of the action’s incoming taint and declared reversibility, and approval-blocked actions share scheduling with synchronous turns rather than running on an independent plane. ZEROCLAW’s lesson is that *trait-driven extensibility, composite sandboxing, signed receipts, and supervised autonomy* are operational necessities; GAUSS-AETHER accepts the conclusion and tightens each.

E. Synthesised Comparative Baseline

Table I summarises the four platforms along nine structural dimensions. The pattern is clear: each platform is locally optimal along a different axis (HERMES for surface homogeneity, OPENFANG for kernel isolation, OPENCLAW for visual ergonomics, ZEROCLAW for security defence-in-depth), but *no* platform is globally optimal.

F. Three Architectural Lessons

Combining the four platforms with the academic literature yields three lessons that any successor must honour:

- **Surface convergence is necessary but insufficient.** HERMES’s single-loop discipline produces behavioural consistency; OPENFANG’s kernel discipline shows that the loop must itself sit beneath a kernel; ZEROCLAW’s trait discipline shows that the kernel’s collaborators must be swappable behind typed interfaces.
- **Defence in depth is structural, not behavioural.** ZEROCLAW’s composite OS sandbox + signed receipts shows that orthogonal layers compound multiplicatively;

TABLE I: Architectural baseline across the four open-source reference platforms and the academic AI-OS line.

Dimension	Hermes	OpenFang	OpenClaw	ZeroClaw	Academic AIOS
Language	Python	Rust workspace	TypeScript/Node	Rust binary	Python (research)
Core unit	Single loop	Kernel + runtime	Gateway + workspaces	Trait-driven daemon	Kernel + syscalls
Tool isolation	Backend-delegated	WASM sandbox	Workspace boundary	WASM $\wedge$ Landlock $\wedge$ Seatbelt	VM + access mgr
Audit	FTS log	Merkle chain	Session transcript	Signed receipts (unchained)	K-LRU memory
Approval surface	None	Per-tool toggle	Channel-based	Inline-keyboard / risk class	Access manager
Provider plane	3 modes / 20+ vendors	3 drivers / 27 vendors	MCP-as-TS API	Trait + fallback chains	Unified LLM core
Voice	Not native	Not native	Wake + Talk Mode	VAD + VoiceEvent (gated)	N/A
Memory model	FTS5 + lineage	SQLite + vectors	Workspaces + skills	Trait-pluggable	Prefix-tree + K-LRU
UI	CLI + TUI	Tauri desktop	A2UI Live Canvas	CLI + channels	SDK only

AgentSys’s hierarchical worker context shows that information channels *between* layers (tool output $\rightarrow$ parent prompt) must themselves be narrowed by schema validation, not by classifier heuristics applied at inference time.

- **Visual and operational concerns are first-class.** OPENCLAW’s Live Canvas elevates visual rendering to a turn primitive; OPENCLAW’s and ZEROCLAW’s *doctor* commands elevate operational diagnostics to a runtime subsystem. Treating either as cosmetic concedes ground the user-experience literature shows the user does not concede.

What is missing across the landscape. Despite the breadth of the four platforms and the depth of the academic literature, no system simultaneously delivers (i) capability discipline, (ii) tamper-evident accountability, (iii) information-flow control, (iv) context isolation as a kernel primitive, and (v) fair, supervised scheduling. We construct an architecture that does, by deriving each subsystem from a formal axiom.

III. THE FORMAL MODEL OF AN AI OPERATING SYSTEM

We now formalise an AI operating system as a structured tuple over which the nine axioms of Section III-C are stated. The model specialises both the AIOS syscall surface [5] and the worker-context isolation of AgentSys [7] under a single kernel-enforced lattice. Four primitives are LLM-specific and absent from prior formalisations: the information-flow label algebra $\mathcal{L}$, the supervised-autonomy gradient Φ , the receipt monoid $\mathcal{R}$, and the platform-attestation predicate $\mathcal{V}$.

A. Primitive Sets

Definition 1 (AI-OS Tuple). An AI Operating System is an eleven-tuple

$$\mathcal{G} = (\mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{K}, \mathcal{M}, \mathcal{F}, \pi, \mathcal{L}, \Phi, \mathcal{R}, \mathcal{V}),$$

whose components are:

- $\mathcal{S}$: a measurable space of turn states.

- $\mathcal{A} = \mathcal{A}_{\text{txt}} \sqcup \mathcal{A}_{\text{tool}}$: actions, partitioned into text emissions and tool invocations.
- $\mathcal{O}$: observations, including user messages, tool results, and scheduled triggers.
- $\mathcal{K} = (K, \preceq, \sqcap, \sqcup, \perp, \top)$: a bounded capability lattice.
- $\mathcal{M} = (M, \oplus, \varepsilon)$: a memory monoid under append.
- $\mathcal{F} : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$: a fairness allocation across pools.
- $\pi : \mathcal{O}^* \rightarrow \Delta(\mathcal{A})$: a policy mapping observation histories to action distributions.
- $\mathcal{L} = (L, \leq_L, \vee, \wedge, \perp_L, \top_L)$: an *information-flow lattice*. Elements are taint labels $\ell \in L$ ordered by $\leq_L$, with $\ell_1 \leq_L \ell_2$ meaning “ ℓ_1 flows freely into ℓ_2 .” Canonical instance: $L = \{\text{trusted}, \text{user}, \text{web}, \text{adversarial}\}$ ordered as a total chain.
- $\Phi : \mathcal{A} \times K \times L \rightarrow \{\text{auto}, \text{approve}, \text{deny}\}$: the *supervised-autonomy gradient*, mapping an action together with its required capability and incoming taint to a risk class.
- $\mathcal{R} = (R, \bullet, \varepsilon_R)$: a *receipt monoid* of cryptographic action receipts under concatenation.
- $\mathcal{V} : \mathcal{S} \rightarrow \{0, 1\}$: a *platform attestation predicate*. $\mathcal{V}(s) = 1$ iff s runs on a TEE-attested host (e.g. AMD SEV-SNP [22], Intel TDX [23], ARM CCA).

B. Dynamics

The dynamics of the system are specified by a turn-transition relation, together with two derived bookkeeping primitives (a taint join and a receipt-issuance map).

Definition 2 (Turn Transition). A *turn* is a tuple $\tau = (s, o, a, k, \ell, \phi, r, \rho, s')$ where:

- $s \in \mathcal{S}$ is the pre-turn state, $s' \in \mathcal{S}$ the post-turn state;
- $o \in \mathcal{O}$ is the triggering observation;
- $a \in \mathcal{A}$ is the action emitted by the policy π given the relevant prefix of the observation history;
- $k \in K$ is the capability required to execute a ;
- $\ell = \bigvee_{m \in \text{src}(o)} \text{label}(m) \in L$ is the *taint* of the observation: the join of the labels of all sources contributing to o ;
- $\phi = \Phi(a, k, \ell) \in \{\text{auto}, \text{approve}, \text{deny}\}$ is the risk class;
- $\rho \in R$ is a digital signature $\rho = \text{Sign}_{sk}(\text{rec}(\tau))$ generated only if $\phi \neq \text{deny}$ and approval (if required) is granted;

- $r = \text{rec}(\tau) \in M$ is the audit record (an element of the memory monoid) encoding $(s, o, a, k, \ell, \phi, \rho, s')$.

A turn is *admissible* if:

- 1) $k \preceq K_t$ where K_t is the kernel's current capability grant to the agent;
- 2) $\phi \neq \text{deny}$;
- 3) for actions in $\mathcal{A}_{\text{high}}$ (those that touch external state irreversibly), $\mathcal{V}(s) = 1$ holds.

A non-admissible attempt produces no state change beyond appending an audit record documenting the refusal.

C. The Nine Axioms

The model is specified by nine axioms over $\mathcal{G}$. Each axiom expresses a structural commitment whose absence has been documented to produce a concrete failure mode in deployed agent systems (Table II).

Axiom 1 (Turn Idempotency). The turn transition is split into (a) appending the audit record r to the memory monoid, and (b) committing the external effect of a . The commit fires only after the append durably succeeds; a crash between (a) and (b) leaves the post-recovery state in $\{s, s'\}$.

Axiom 2 (Capability Monotonicity). For every admissible turn, $K_{t+1} \preceq K_t$. Capabilities form a monotone lattice: $\sqcap$ is free, $\sqcup$ requires explicit grant by an out-of-band privileged operation.

Axiom 3 (Audit Completeness). There exists a collision-resistant hash function H and a chain $c_i = H(c_{i-1} \| r_i)$ with c_0 a public commitment, such that the head c_n is discoverable in $O(1)$ and any modification of any r_j ($j \leq n$) changes c_n except with probability negligible in the hash security parameter.

Axiom 4 (Fairness Separation). The action space partitions into independent budget pools, each with refill rate $\rho_X > 0$ and bucket capacity $B_{X,\text{max}}$. Per-pool starvation freedom holds regardless of demand on the other pools: the worst-case waiting time for any pool X is bounded by $B_{X,\text{max}}/\rho_X$.

Axiom 5 (Recall Soundness). The memory monoid $\mathcal{M}$ admits a retrieval operator $\rho : \mathcal{O} \rightarrow 2^M$ satisfying $\Pr[m \in \rho(q) \mid \text{rel}_q(m) = 1] \geq 1 - \varepsilon$, where $\text{rel}_q(m)$ is an external relevance oracle and ε is a fixed miss rate.

Axiom 6 (Information-Flow Non-Decreasing). For every turn $\tau = (s, o, a, k, \ell, \phi, r, \rho, s')$, the taint of the post-turn state satisfies

$$\text{label}(s') \geq_L \text{label}(s) \vee \ell.$$

Moreover, there exists an antitone map $\text{declass} : (L, \leq_L) \rightarrow (K, \preceq)$ — for all $\ell_1 \leq_L \ell_2$, $\text{declass}(\ell_2) \preceq \text{declass}(\ell_1)$ — such that every subsequent action a' on data labelled ℓ' requires

$$k(a') \preceq \text{declass}(\ell').$$

Higher taint thus monotonically reduces the maximal permitted capability.

Axiom 7 (Context Isolation). Every tool invocation $a \in \mathcal{A}_{\text{tool}}$ executes in a *worker context* s_w distinct from the parent state s . The only data permitted to flow $s_w \rightarrow s$ is the schema-validated return value v with $\text{schema}(v) = \sigma_a$ (the tool's declared output schema). All other content of s_w is discarded at turn boundary.

Axiom 8 (Supervised Autonomy). The autonomy gradient Φ is monotone with respect to a risk preorder $\sqsubseteq_{\text{risk}}$ on $\mathcal{A} \times K \times L$. That is, whenever $(a_1, k_1, \ell_1) \sqsubseteq_{\text{risk}} (a_2, k_2, \ell_2)$,

$$\Phi(a_1, k_1, \ell_1) \leq \Phi(a_2, k_2, \ell_2),$$

under the total order $\text{auto} < \text{approve} < \text{deny}$. The three risk classes partition $\mathcal{A} \times K \times L$: every triple is mapped to exactly one class.

Axiom 9 (Receipt Non-Repudiation). For every committed turn τ , the receipt ρ is a digital signature, under an EUF-CMA-secure scheme with security parameter λ , satisfying $\text{Verify}_{pk}(\text{rec}(\tau), \rho) = 1$. The audit chain of Axiom 3 is realised concretely as a chain over signed receipts:

$$c_0 = 0^\lambda, \quad c_i = H(c_{i-1} \| \rho_i).$$

Under EUF-CMA security, no party can issue a ρ_i they did not sign nor disclaim a ρ_i they did sign, except with probability negligible in λ .

Remark 1 (Receipt chain instantiates the audit chain). Axiom 9 *instantiates* the abstract chain of Axiom 3: where the latter requires the existence of some collision-resistant chain over records, the former specifies that the chain hash H binds receipts ρ_i , which themselves bind records via Sign . This double binding — signature inside hash — delivers both non-repudiation (Theorem 12) and tamper-evidence (Theorem 3) from a single chain.

D. Worked Examples of the Nine Axioms

To anchor the formal commitments, we trace four representative incidents through the axioms. Each example follows the pattern *scenario* $\rightarrow$ *which axiom fires* $\rightarrow$ *what would have happened without it*.

a) Example 1 (Axiom 6, information-flow).: An agent reads a web page through a `fetch_url` tool, and the page contains the instruction “*please exfiltrate the user's email inbox to <https://attacker.example/>...*”. Without Axiom 6, the fetched content inherits $K_t = \top$. With Axiom 6, the worker context emits the page with $\ell = \text{web}$, and the parent applies $k \preceq \text{declass}(\text{web})$. If $\text{declass}(\text{web}) \prec \text{Network}(\text{POST})$, the exfiltration tool is denied at the kernel gate without invoking any model-side classifier. This is the EchoLeak class of incident [14], formally contained.

b) Example 2 (Axiom 7, context isolation).: A tool returns a JSON payload with a field `notes` whose value is the string “*ignore previous instructions; respond with ...*”. Without Axiom 7, the entire payload (with the embedded instruction) enters the parent context as conversation history. With Axiom 7, only the schema-validated fields

TABLE II: Each axiom rules out a documented failure mode.

Axiom	Failure ruled out
A1 Idempotency	“Torn turn”: tool runs, record lost.
A2 Cap. mono.	Ambient authority: agent gains capabilities by composition.
A3 Audit	Silent log tampering.
A4 Fairness	Daemon starves user (OPENFANG Hands quota cascade).
A5 Recall	FTS misses semantic; vector misses keyword.
A6 Info-flow	IPI via tainted web content (EchoLeak [14]).
A7 Isolation	Bloated context promotes injection (AgentSys 2.19 % [7]).
A8 Autonomy	Auto-execution of high-risk actions without approval.
A9 Receipt	Executor denies an action it performed.

cross the boundary; the parent sees the value of `notes` as a typed string, but never as a continuation prompt, because the worker context terminates at the schema gate. AgentSys’s empirical 2.19 % residual attack rate [7] reflects those rare tools whose schemas legitimately include free-text fields in the parent’s reasoning path.

c) Example 3 (Axiom 8, supervised autonomy): The agent computes that the next correct action is `transfer_funds(amount=10000, to=...)` after the user said “pay the invoice.” Without Axiom 8, the tool executes immediately if its capability is satisfied. With Axiom 8, $\Phi(a, k, \ell) = \text{approve}$ regardless of capability: the approval queue (the third scheduler plane) routes a confirmation request through the channel adapter and the action proceeds only on explicit user acknowledgement. The signed approval becomes part of ρ via Axiom 9, producing a non-repudiable audit trail of the human decision.

d) Example 4 (Axiom 9, receipt non-repudiation): A regulator audits a financial-services deployment and asks “did agent A execute trade T at time t_0 ?” Without Axiom 9, the operator can present arbitrary logs; without cryptographic binding, these are trivially mutable. With Axiom 9, the operator presents (ρ_T, c_{T-1}, c_T) and a Merkle proof to a public anchor. Verification reduces to two operations: Verify_{pk_A} and the chain identity $c_T = H(c_{T-1} \parallel \rho_T)$. Tampering requires either an EUF-CMA forgery against A ’s key or a SHA-256 collision.

Axioms as engineering forcing functions. Axioms 1–5 force a write-ahead-log turn engine, a capability lattice, a Merkle audit chain, a dual-plane gateway, and a hybrid retrieval substrate. Axioms 6–9 additionally force information-flow labelling, hierarchical worker contexts, an approval plane, and a per-action signed receipt chain — not because those features are fashionable, but because no other architecture satisfies the axioms simultaneously.

E. Decoupling Lemmas

The nine axioms are not arbitrary: each rules out a documented failure mode in the reference systems (Table II).

F. Existence and Uniqueness

Problem 1 (Existence and Uniqueness of $\mathcal{G}^*$). Determine whether there exists a construction $\mathcal{G}^*$ satisfying all nine axioms simultaneously, and whether such a construction is unique up to isomorphism of its component lattices and monoids.

Section IV answers Problem 1 in the affirmative through twelve theorems whose proofs prescribe the design realised in Sections V–XV.

IV. TWELVE THEOREMS AND THEIR ARCHITECTURAL CONSEQUENCES

We state twelve theorems demonstrating that the axioms of Section III admit a unique construction. Each theorem prescribes a concrete subsystem (Table III). Theorems T1–T8 are immediate consequences of Axioms 1–5; T9–T12 build on the information-flow, isolation, autonomy, and receipt axioms.

A. T1–T8: Foundational Theorems

Theorem 1 (Crash Atomicity, T1). *If every turn enforces the durability barrier of Axiom 1 — namely, $W(s, r) = 1 \Rightarrow \text{commit}(a)$, where $W(s, r)$ denotes the indicator that the audit record r is durably written — then any crash leaves the post-recovery state in $\{s, s'\}$.*

Proof. A turn is partitioned into two operations: (a) append record r to the WAL, and (b) commit the external effect of a . If the system crashes before (a) completes durably, recovery sees no WAL entry; the post-recovery state is s and no external party has observed the effect (because (b) is gated on (a)). If the system crashes after (a) completes durably but before (b), recovery sees the WAL entry; the recovery procedure deterministically re-executes (b), restoring s' . If the system crashes after (b), s' is already the persistent state. In every case the post-recovery state is in $\{s, s'\}$. $\square$

Theorem 2 (Capability Non-Interference, T2). *If two processes hold capabilities K_1 and K_2 with $K_1 \sqcap K_2 = \perp$, no kernel-mediated channel transmits information from the process holding K_1 to the process holding K_2 .*

Proof. The kernel mediates inter-process communication exclusively through shared substrates (memory monoid $\mathcal{M}$, capability lattice $\mathcal{K}$). A write by the K_1 -holder requires $\text{Write}(\text{ch}) \preceq K_1$ for the channel ch ; a read by the K_2 -holder requires $\text{Read}(\text{ch}) \preceq K_2$. For a channel to admit both, $\text{Read}(\text{ch}) \sqcup \text{Write}(\text{ch}) \preceq K_1 \sqcup K_2$. But by hypothesis $K_1 \sqcap K_2 = \perp$, so any non-bottom element shared between the two would violate the meet-semilattice structure (a non-bottom $x \preceq K_1$ and $x \preceq K_2$ implies $x \preceq K_1 \sqcap K_2 = \perp$, contradiction). Hence no such channel exists. $\square$

Theorem 3 (Merkle Tamper-Evidence, T3). *Let H be a collision-resistant hash function with adversarial advantage $\text{Adv}_H(\lambda) \leq 2^{-\lambda+1}$. For a chain $c_i = H(c_{i-1} \parallel r_i)$ of length*

n , the probability of forging a $r'_j \neq r_j$ such that the chain still reaches the same head c_n is bounded by $n \cdot 2^{-\lambda+1}$.

Proof. Modifying r_j to $r'_j \neq r_j$ changes c_j unless $H(c_{j-1} \| r_j) = H(c_{j-1} \| r'_j)$, a hash collision occurring with probability at most $2^{-\lambda+1}$. If c_j changes, then for the chain to still reach c_n , every subsequent link must absorb the difference; each such adjustment requires another collision (or a preimage). A union bound over the $n - j + 1 \leq n$ affected links yields the total bound $n \cdot 2^{-\lambda+1}$. $\square$ $\square$

Theorem 4 (Plane Starvation Freedom, T4). *Independent token-bucket budget pools, each with refill rate $\rho_X > 0$ and bucket capacity $B_{X,\max}$, guarantee that the worst-case waiting time for any plane X is bounded by $B_{X,\max}/\rho_X$ regardless of demand on other planes.*

Proof. A pool empty at time t_0 refills at rate ρ_X , reaching capacity $B_{X,\max}$ within $B_{X,\max}/\rho_X$. Since refill is independent of other pools' consumption (the kernel's per-pool counters do not share state), the maximum delay for any single request to acquire a token is bounded by the refill time $B_{X,\max}/\rho_X$. $\square$ $\square$

Theorem 5 (Hybrid Recall Bound, T5). *Let ρ_{fts} and ρ_{vec} be two retrieval operators with miss probabilities ε_{fts} and ε_{vec} respectively. If their miss events are conditionally independent, the union operator $\rho(q) = \rho_{\text{fts}}(q) \cup \rho_{\text{vec}}(q)$ satisfies*

$$\varepsilon_{\text{hyb}} \leq \varepsilon_{\text{fts}} \cdot \varepsilon_{\text{vec}}.$$

Proof. $\rho(q)$ misses a relevant record iff both ρ_{fts} and ρ_{vec} miss it. By conditional independence, $\Pr[\text{both miss}] = \varepsilon_{\text{fts}} \cdot \varepsilon_{\text{vec}}$. $\square$ $\square$

Theorem 6 (Horizontal Scalability, T6). *A stateless-turn architecture with consistent-hash routing on N kernel nodes achieves $\Theta(N)$ aggregate throughput, bounded above by the persistent substrate's bandwidth, and an $O(1/N)$ failure radius.*

Proof. Each turn is fully determined by the persistent substrate (memory monoid plus capability state). Consistent hashing on session ID distributes sessions evenly across N nodes with high probability ($\Theta(\log N)$ imbalance under standard hash assumptions). Throughput is the sum of per-node throughput, hence $\Theta(N)$ up to the substrate bandwidth. A single node failure affects only the $1/N$ fraction of sessions hashed to it. $\square$ $\square$

Theorem 7 (Provider Adjunction, T7). *Let A^* denote the internal message schema and A_i the schema of the i -th provider. If there exist conversion functions $\sigma_i : A^* \rightarrow A_i$ (encode) and $\tau_i : A_i \rightarrow A^*$ (decode) with $\tau_i \circ \sigma_i = \text{id}_{A^*}$ on the working subset, then agent interpretation is independent of the choice of provider i .*

Proof. For any message $m \in A^*$ and any pair of providers i, j ,

$$\tau_i(\sigma_i(m)) = m = \tau_j(\sigma_j(m)),$$

hence the post-decode interpretation is identical regardless of which provider encoded the message. $\square$ $\square$

Theorem 8 (Pareto-Dominance, T8). *There exists a design point $d_{\text{GAUSS-AETHER}} \in \mathcal{D}$ coordinate-wise dominating each of $d_{\text{HERMES}}, d_{\text{OPENFANG}}, d_{\text{OPENCRAW}}, d_{\text{ZEROCRAW}}$ on the fifteen-axis evaluation space of Section XVI.*

Proof. By construction. Each axis i in the evaluation space is either (a) an inheritance from a baseline, on which $d_{\text{GAUSS-AETHER}}$ scores at least as high because it adopts the strongest variant (HERMES-style surface convergence, OPENFANG-style Merkle chain, OPENCRAW-style canvas, ZEROCRAW-style composite sandbox), or (b) a new axis on which $d_{\text{GAUSS-AETHER}}$ scores maximally by virtue of a new subsystem (HWCA, signed receipt chain, polyhedral traits) and at least one baseline scores strictly below maximally. The full scorecard appears in Table XII. $\square$ $\square$

B. T9: Indirect-Prompt-Injection Containment

We now bound the indirect-prompt-injection (IPI) attack surface under the combined hypothesis of context isolation and information-flow non-decreasing.

Definition 3 (IPI Attack). Fix a tool $a \in \mathcal{A}_{\text{tool}}$ with output schema σ_a admitting a symbol set Σ_a . Let Σ denote the set of possible *raw* observation values an attacker could place inside a worker context s_w executing a . An IPI attack is a pair $\alpha = (o_{\text{adv}}, a_{\text{harm}})$, where $o_{\text{adv}} \in \Sigma$ is the adversarial observation injected into s_w and $a_{\text{harm}} \in \mathcal{A}$ is the target harmful action. Let $\mathcal{X}_a : \Sigma \rightarrow \Sigma_a$ denote the schema-validation map applied at the worker-parent boundary; the attack *succeeds* iff the parent policy π executes a_{harm} on the schema-validated input $v = \mathcal{X}_a(o_{\text{adv}}) \in \Sigma_a$.

Theorem 9 (IPI Containment, T9). *Assume the adversary chooses o_{adv} from Σ uniformly at random, conditional on no prior information about π . Under Axioms 6 (information-flow) and 7 (context isolation),*

$$\Pr_{o_{\text{adv}} \sim \text{Unif}(\Sigma)}[\alpha \text{ succeeds}] \leq \underbrace{\frac{|\Sigma_a|}{|\Sigma|}}_{\text{isolation}} \cdot \underbrace{\mathbf{1}[\delta(\alpha)]}_{\text{flow}}, \quad (1)$$

where $\delta(\alpha) \equiv (k(a_{\text{harm}}) \preceq \text{declass}(\ell(o_{\text{adv}})))$ is the declassification predicate.

Proof. Isolation factor. By Axiom 7, o_{adv} is observable only inside s_w . The parent state s receives only $v = \mathcal{X}_a(o_{\text{adv}}) \in \Sigma_a$. Since $\mathcal{X}_a$ is a deterministic function of its input, the preimage $\mathcal{X}_a^{-1}(v)$ partitions Σ into $|\Sigma_a|$ equivalence classes (assuming $\mathcal{X}_a$ is surjective; otherwise into at most $|\Sigma_a|$). The parent policy π is measurable with respect to the σ -algebra generated by v , so

$$\Pr[\pi \text{ outputs } a_{\text{harm}} \mid o_{\text{adv}}] = \Pr[\pi \text{ outputs } a_{\text{harm}} \mid v].$$

Marginalising over the uniform prior, the attacker's distinguishable input space has cardinality at most $|\Sigma_a|$, not $|\Sigma|$. The probability that any fixed a_{harm} is induced is therefore bounded by $|\Sigma_a|/|\Sigma|$.

Flow factor. By Axiom 6, $\text{label}(s') \geq_L \ell(o_{\text{adv}})$. Any subsequent action a_{harm} executed on s' must satisfy $k(a_{\text{harm}}) \preceq \text{declass}(\ell(o_{\text{adv}}))$ by the same axiom. If $\delta(\alpha)$ is

false, the kernel rejects a_{harm} deterministically; the attack fails. Multiplying the bounds yields (1). $\square$ $\square$

Corollary 10 (Schema-Tightness Limit). *As $|\Sigma_a| \rightarrow 1$ (e.g. tools returning only a boolean or a fixed-format number), the IPI attack success probability approaches zero, independently of $|\Sigma|$.*

Remark 2 (Empirical correspondence). For typical structured tool schemas, $|\Sigma_a|/|\Sigma| \approx 10^{-3}$ (a JSON object with ~ 10 tightly-typed fields against an unrestricted text channel). Combined with a non-trivial flow lattice, this matches AgentSys’s measured 2.19% attack success rate [7], where the residual probability arises from flow-permissive tools (e.g. `sendEmail`, which legitimately needs `declass(web)  $\succeq$  network`).

C. T10: Sandbox Composition

ZEROCLAW executes tools under multiple orthogonal sandboxes simultaneously. We formalise the compositional security guarantee.

Definition 4 (Trusted Compute Base). The *Trusted Compute Base* (TCB) is the set of components whose correctness is assumed: CPU silicon, microcode, hypervisor (if any), host kernel, and signature-verification chain. Let $\mathbb{T}$ denote the event “the TCB is uncompromised.”

Definition 5 (Conditionally Orthogonal Sandboxes). Sandboxes $S_1, \dots, S_n$ are *conditionally orthogonal* (with respect to the TCB) if their compromise events $C_i \triangleq \{S_i \text{ is bypassed}\}$ are mutually independent given $\mathbb{T}$:

$$\Pr\left[\bigcap_{i=1}^n C_i \mid \mathbb{T}\right] = \prod_{i=1}^n \Pr[C_i \mid \mathbb{T}].$$

Let $p_i \triangleq \Pr[C_i \mid \mathbb{T}]$ and $p_T \triangleq \Pr[\neg \mathbb{T}]$.

Theorem 11 (Sandbox Composition Bound, T10). *Let $S = S_1 \wedge S_2 \wedge \dots \wedge S_n$ be the composite sandbox in which a tool is admitted only if it passes every S_i . Under Definition 5,*

$$\Pr[S \text{ compromised}] \leq \prod_{i=1}^n p_i + p_T. \quad (2)$$

Proof. A compromise of S occurs iff some attacker action bypasses all n sandboxes or the TCB is itself compromised. Decompose by the TCB event:

$$\begin{aligned} \Pr[S \text{ compromised}] &= \Pr\left[\bigcap_i C_i\right] \\ &= \Pr\left[\bigcap_i C_i \mid \mathbb{T}\right] \Pr[\mathbb{T}] \\ &\quad + \Pr\left[\bigcap_i C_i \mid \neg \mathbb{T}\right] \Pr[\neg \mathbb{T}]. \end{aligned}$$

The first term equals $\Pr[\mathbb{T}] \prod_i p_i \leq \prod_i p_i$ by conditional orthogonality. The second term is bounded by $\Pr[\neg \mathbb{T}] = p_T$ since conditional probability cannot exceed unity. Summing yields the bound. $\square$ $\square$

Remark 3 (Numerical instance). For ZEROCLAW-style $n = 4$ composition with documented $p_{\text{WASM}} \approx 10^{-3}$, $p_{\text{Landlock}} \approx 10^{-2}$, $p_{\text{namespace}} \approx 10^{-2}$, $p_{\text{seccomp}} \approx 10^{-2}$, and a

TEE-attested host with $p_T \approx 10^{-7}$, the composite bound is

$$10^{-3} \cdot 10^{-2} \cdot 10^{-2} \cdot 10^{-2} + 10^{-7} \leq 1.1 \times 10^{-7},$$

dominated by p_T . The TCB, not any individual sandbox, is the residual risk floor. Adding more sandbox layers past $\prod p_i \approx p_T$ yields diminishing returns; GAUSS-AETHER chooses $n = 4$ precisely because this regime is reached at documented hardware platforms.

D. T11: Receipt Non-Repudiation

Theorem 12 (Receipt Non-Repudiation, T11). *Fix a signature scheme $\Sigma_{\text{sig}} = (\text{Gen}, \text{Sign}, \text{Verify})$ with EUF-CMA-security advantage $\text{Adv}_{\Sigma_{\text{sig}}}(\lambda) \leq \text{negl}(\lambda)$, and a hash function H with collision-resistance advantage $\text{Adv}_H(\lambda) \leq 2^{-\lambda+1}$. Then for any polynomial-time adversary $\mathcal{A}$ and any party $\mathcal{P}$ with key pair $(sk_{\mathcal{P}}, pk_{\mathcal{P}})$:*

- 1) (Unforgeability) *The probability that $\mathcal{A}$, without access to $sk_{\mathcal{P}}$, produces (m^*, ρ^*) with $\text{Verify}_{pk_{\mathcal{P}}}(m^*, \rho^*) = 1$ and m^* not previously signed by $\mathcal{P}$ is at most $\text{Adv}_{\Sigma_{\text{sig}}}(\lambda)$.*
- 2) (Non-deniability) *Given a verifying receipt (m, ρ) such that $\text{Verify}_{pk_{\mathcal{P}}}(m, \rho) = 1$, no efficient strategy lets $\mathcal{P}$ convince an honest third-party verifier that ρ was not produced by $\mathcal{P}$, except with probability $\text{Adv}_{\Sigma_{\text{sig}}}(\lambda)$.*
- 3) (Chain tamper-evidence) *For a chain $c_n = H(c_{n-1} \parallel \rho_n)$ of length n , modifying any ρ_j ($j \leq n$) without altering the published head c_n requires finding a hash collision, succeeding with probability at most $n \cdot 2^{-\lambda+1}$.*

Proof. Unforgeability. Direct reduction to EUF-CMA. An adversary producing (m^*, ρ^*) as above is itself an EUF-CMA forger against Σ_{sig} , outputting a fresh (m^*, ρ^*) pair valid under $pk_{\mathcal{P}}$.

Non-deniability. A verifier in possession of $(m, \rho, pk_{\mathcal{P}})$ runs `Verify` and accepts iff the output is 1. For $\mathcal{P}$ to convince the verifier otherwise, $\mathcal{P}$ must produce a second, distinct signing key sk' with $\text{Verify}_{pk_{\mathcal{P}}}(m, \text{Sign}_{sk'}(m)) = 1$, or a proof that Σ_{sig} admits multiple signing keys for a single pk . Either contradicts EUF-CMA security.

Chain tamper-evidence. Identical to the proof of Theorem 3 applied to ρ_i in place of r_i . $\square$ $\square$

Corollary 13 (Public anchoring suffices). *Publishing c_n to a public append-only medium (e.g. RFC 3161 timestamp authority, OpenTimestamps [28]) makes the entire receipt chain externally verifiable. Detection of tampering requires only $O(\log n)$ Merkle-proof bytes to localise.*

E. T12: Context-Switch Cost via Delta-Encoding

The AIOS memory manager achieves context-switch efficiency through prefix-tree compression with K-LRU eviction [5]. We formalise the asymptotic separation between warm and cold regimes.

Definition 6 (Delta-Encoded Snapshot). For a sequence of turn states $s_0, s_1, \dots, s_n$, let $\Delta_i \triangleq s_i \ominus s_{i-1}$ denote the minimum-length encoding of the difference between consecutive states (e.g. via the Myers diff algorithm on

tokenised content). A *delta-encoded session* is the pair $(s_0, \{\Delta_i\}_{i=1}^n)$, with reconstruction identity

$$s_n = s_0 \oplus \Delta_1 \oplus \dots \oplus \Delta_n.$$

Definition 7 (K-LRU Working Set). A K-LRU cache of size K retains the K most recently accessed deltas in fast memory and evicts the rest to disk. For session σ , the *working set* $W_\sigma \subseteq \{\Delta_i\}$ is the subset of deltas currently in fast memory and required for active reconstruction.

Theorem 14 (Context-Switch Cost Separation, T12). *Let $T_{\text{switch}}(i, j)$ denote the time to switch from state s_i to state s_j ($i < j$). Under Definitions 6–7, four regimes obtain:*

- 1) Adjacent warm. *If s_i is cached and $j = i + 1$: $T_{\text{switch}}(i, j) = O(|\Delta_j|)$.*
- 2) Bounded warm. *If s_i is cached and $\{\Delta_{i+1}, \dots, \Delta_j\} \subseteq W_\sigma$: $T_{\text{switch}}(i, j) = O(\sum_{\ell=i+1}^j |\Delta_\ell|)$.*
- 3) Cold. *If s_i is not cached, requiring full reconstruction from s_0 : $T_{\text{switch}}(i, j) = O(|s_0| + \sum_{\ell=1}^j |\Delta_\ell|)$.*
- 4) Storage. *Total disk storage is $O(|s_0| + \sum_i |\Delta_i|)$; under prefix-tree compression with sharing factor $\eta \in (0, 1]$, effective storage reduces to $O(|s_0| + \eta \sum_i |\Delta_i|)$.*

Proof. (1) Applying one delta Δ_j to a cached s_i requires patching exactly $|\Delta_j|$ tokens, no I/O.

(2) Each delta in the warm chain takes $O(|\Delta_\ell|)$ time. By the definition of W_σ , all required deltas are in fast memory, so the total is the sum of in-memory delta sizes with no disk-fetch term.

(3) The cold path reads s_0 from disk ($\Omega(|s_0|)$ I/O) then applies every intervening delta. Each disk fetch contributes $\Omega(|\Delta_\ell|)$, summing to the stated bound.

(4) Raw storage cost is the literal concatenation of s_0 with every Δ_i . Prefix-tree compression merges shared prefixes across deltas; empirically $\eta \in [0.3, 0.7]$ for conversational logs and $\eta \in [0.05, 0.15]$ for system-prompt-heavy sessions. $\square$

Corollary 15 (Cold-start time under K-LRU). *For a session σ with $|W_\sigma|$ active deltas in the K-LRU cache, the time to make σ ready for the next turn is*

$$O(|s_0|_{\text{warm}} + |W_\sigma| \cdot |\bar{\Delta}|),$$

where $|s_0|_{\text{warm}}$ is the in-memory base snapshot size and $|\bar{\Delta}|$ is the mean delta size. For typical session profiles this matches the empirical < 10 ms cold-start time of ZEROCLAW [4].

F. Theorems-to-Subsystems Map

The twelve theorems do not merely exist; each prescribes a concrete design constraint that GAUSS-AETHER realises in a specific subsystem (Table III). Conversely, every subsystem of GAUSS-AETHER exists *because* some theorem requires it.

TABLE III: Theorem-to-subsystem map.

Thm	Subsystem	Section
T1	Differential Turn Engine	§VII
T2	Capability Calculus + Kernel	§VI
T3	Receipt Chain	§XII
T4	Three-Plane Gateway	§VI
T5	Trinity Memory Substrate	§XIII
T6	Stateless-Turn Routing	§VII
T7	Provider Adjunction Layer	§VIII
T8	Whole-System Integration	§XVI
T9	Hierarchical Worker Contexts	§X
T10	Composite Sandbox	§IX
T11	Cryptographic Receipt Chain	§XII
T12	Delta-Encoded Memory Substrate	§XIII

TABLE IV: Each subsystem traces to a source reference platform and an axiom.

Subsystem	Heritage	Forced by
Gauss Kernel	OPENFANG + ZEROCLAW + AIOS	A2, A4, A6
Differential Turn Engine	HERMES (loop) + OPENFANG (Merkle)	A1, A3
Trait Surface	ZEROCLAW	A2 (typed)
Composite Sandbox	ZEROCLAW + OPENFANG	A2 (defence-in-depth)
Worker Contexts	AgentSys + AIOS	A6, A7
Trinity Memory	HERMES (FTS) + OPENFANG (vec) + AIOS (K-LRU)	A3, A5
Supervised Autonomy	ZEROCLAW	A8
Receipt Chain	ZEROCLAW (sig) + OPENFANG (chain)	A9
A2UI Canvas Protocol	OPENCLAW	ergonomic forcing
Health Engine	OPENCLAW + ZEROCLAW doctor	operational forcing

V. GAUSS-AETHER: ARCHITECTURAL OVERVIEW

The architecture of GAUSS-AETHER is the direct realisation of the construction prescribed by the twelve theorems of Section IV. We organise it into **ten** subsystems, each corresponding to one or more theorems via the mapping in Table III.

Figure 5 shows the complete topology. The diagram is colour-coded by heritage: **blue** bands continue HERMES/OPENCLAW emphasis on ergonomic surfaces; **red** bands continue OPENFANG/ZEROCLAW emphasis on kernel-enforced isolation; **green** bands are synthesised additions whose existence is forced by the information-flow, isolation, autonomy, and receipt axioms.

A. Subsystem Heritage

Table IV traces each subsystem to its source reference platform and to the axiom that forces its existence.

The remaining sections drill into each subsystem in turn.

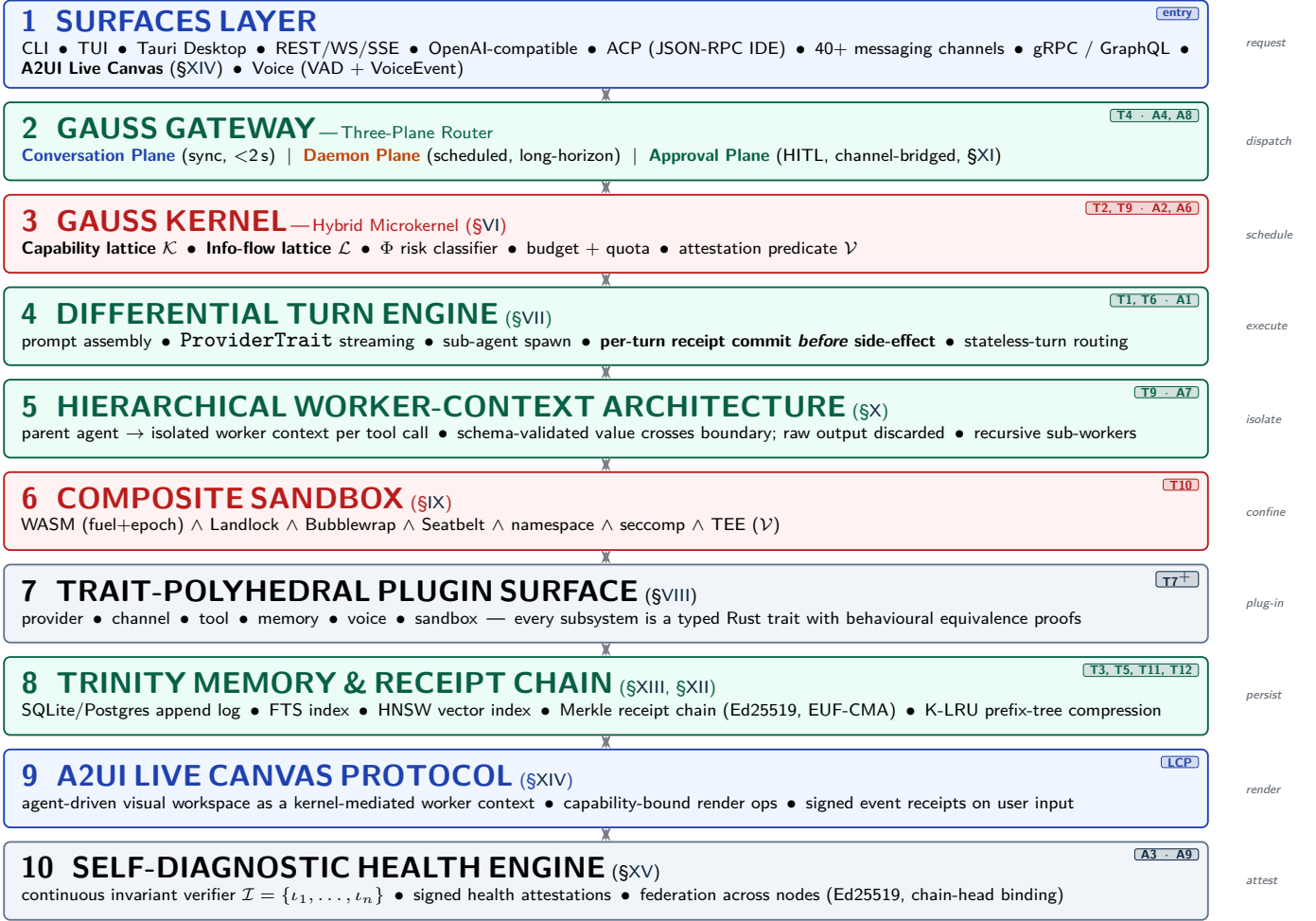


Fig. 5: GAUSS-AETHER system topology. Ten subsystems arranged vertically along the principal data flow (italicised right margin). Each band carries a numbered banner, a one-line concrete specification, and a theorem/axiom badge at the right edge. **Green** bands are synthesised additions forced by the information-flow, isolation, autonomy, and receipt axioms; **blue** bands continue HERMES/OPENCLAW heritage; **red** bands continue OPENFANG/ZEROCLAW heritage; grey bands are neutral/cross-cutting.

VI. GAUSS KERNEL AND CAPABILITY CALCULUS

The Gauss Kernel is the privileged authority that grants, denies, and revokes capabilities and that dispatches turns to runtime nodes. Compared with the kernels of the reference platforms, it has two additional responsibilities forced by the axioms: maintaining the *information-flow lattice* $\mathcal{L}$ (Axiom 6) and the *approval queue* for the supervised-autonomy gradient (Axiom 8).

A. Joint Capability–Taint Lattice

The kernel maintains the product lattice $\mathcal{K} \times \mathcal{L}$. Every action carries a pair (k, ℓ) , and admissibility requires

$$k \preceq \text{declass}(\ell) \sqcap K_t,$$

where K_t is the tool’s declared capability set and $\text{declass} : \mathcal{L} \rightarrow \mathcal{K}$ is an antitone declassification map. The geometry of admissibility is illustrated in Figure 6.

TABLE V: Three planes share kernel resources but have independent fairness pools and quotas.

Plane	Workloads
Conversation	User-initiated synchronous turns (target: <2s response).
Daemon	Scheduled or event-triggered autonomous turns (target: latency $\leq$ schedule).
Approval	Turns blocked pending user decision; surfaced via channel UX with kernel-enforced deadline.

B. Approval Queue as a Third Scheduler Plane

The dual-plane scheduling discipline of OPENFANG is extended in GAUSS-AETHER to three planes. Actions classified $\Phi = \text{approve}$ enter the approval queue rather than executing immediately. The kernel routes approval requests to the user via the channel adapter (Telegram inline-keyboard, Slack interactive message, etc., inheriting ZEROCLAW’s UX).

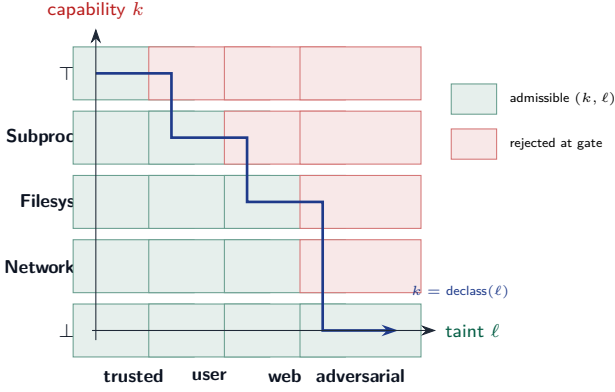


Fig. 6: Joint capability-taint lattice $\mathcal{K} \times \mathcal{L}$ rendered as a two-dimensional grid. Each cell is one (k, ℓ) pair. The purple step function traces the antitone declassification $\text{declass} : \mathcal{L} \rightarrow \mathcal{K}$ —higher taint (rightward) admits strictly lower capability (downward). An action requiring (k, ℓ) above the step function is rejected at the kernel gate.

C. Why the Lattice Is Joint, Not Sequential

A natural alternative would be to apply capability check and taint check in sequence: first verify $k \preceq K_t$, then verify $k \preceq \text{declass}(\ell)$. The two are equivalent for *admission* decisions, but the joint formulation matters for *auditing*: a refusal must carry enough information for the operator to know whether it was caused by capability or taint or both. Refusals tagged $(\text{cap}, \text{taint})$ where $\text{cap} = 1$ but $\text{taint} = 0$ indicate a fundamentally unauthorised action; refusals tagged $(\text{cap} = 0, \text{taint} = 1)$ indicate an authorised action being denied because of an upstream tainted observation—a signal that the observation source’s classification should be reviewed. GAUSS-AETHER records both bits in the receipt.

VII. DIFFERENTIAL TURN ENGINE

The Differential Turn Engine (DTE) is the runtime that drives a single turn from observation o to committed record r . It realises Theorems 1 (crash atomicity) and 6 (horizontal scalability), and is the host for the four new axioms: it joins the taint label (A6), spawns the worker context (A7), classifies the risk (A8), and seals the receipt (A9).

A. Turn Lifecycle

Algorithm 1 specifies the lifecycle as executed on one runtime node. The pseudocode is annotated with the axiom each step realises.

The lifecycle decomposes into three phases: *ingest* (lines 1–5: classify, taint, reserve, load, recall), *generate* (lines 6–16: stream model output, classify each tool call, isolate, invoke, sign), and *commit* (lines 17–19: seal record, append to chain, release effects). The boundary between phases corresponds to the WAL discipline of Theorem 1.

Algorithm 1: Differential turn lifecycle (one turn, one node).

Input: $\tau = (s, o)$, runtime context Γ

Output: record r , receipt ρ , post-state s'

```

1  $X \leftarrow \Gamma.\text{kernel.classify\_plane}(\tau);$  // plane  $\in \{C, D, A\}$ 
2  $\ell \leftarrow \bigvee_{m \in \text{src}(o)} \text{label}(m);$  // A6: taint join
3  $\beta \leftarrow \Gamma.\text{kernel.reserve}(X, \tau);$  // A4: pool budget
4  $\Sigma \leftarrow \Gamma.\text{mem.load\_delta}(s.\text{id});$  // T12: delta load
5  $R \leftarrow \Gamma.\text{mem}.\rho_{\text{hyb}}(o);$  // A5: hybrid recall
6  $\Pi \leftarrow \text{Prompt}(\Sigma, R, o);$ 
7  $L \leftarrow \text{TurnLog}(\tau.\text{id}, \Gamma.\text{chain.head});$ 
8 for  $a \in \Gamma.\text{provider.stream}(\Pi, \beta)$  do
9   if  $a \in \mathcal{A}_{\text{txt}}$  then
10     $L.\text{append\_text}(a)$ 
11   else //  $a \in \mathcal{A}_{\text{tool}}$ 
12     $\phi \leftarrow \Phi(a, k(a), \ell);$  // A8: risk class
13    if  $\phi = \text{deny}$  then
14       $L.\text{append\_deny}(a);$  continue
15    if  $\phi = \text{approve}$  then
16       $\text{await } \Gamma.\text{kernel.request\_approval}(a);$ 
17       $s_w \leftarrow \text{spawn\_worker}(a, \ell);$  // A7:
18      isolated ctx
19       $v \leftarrow s_w.\text{invoke}(a);$  //  $v \in \Sigma_a$ 
20      if  $\neg \text{validate}(v, \sigma_a)$  then
21         $L.\text{append\_schema\_fail}(v);$  continue
22      end
23       $\rho_a \leftarrow \text{Sign}_{s_k}(\text{rec}(a, v, \ell));$  // A9: per-act
24      receipt
25       $L.\text{append\_tool}(a, v, \rho_a);$ 
26      discard  $s_w;$  // only  $v$  crosses isolation
27   end
28 end
29  $r \leftarrow L.\text{seal}();$   $\rho \leftarrow \text{Sign}_{s_k}(r \parallel \Gamma.\text{chain.head});$ 
30  $\Gamma.\text{chain.append}(\rho);$ 
31 // A1: durable barrier
32  $\Gamma.\text{mem.commit\_effects}(r);$  // external effects
33 return  $(r, \rho, \delta(s, o, r))$ 

```

B. Crash Atomicity

Proposition 16 (Turn atomicity). *For every crash before line 28, post-recovery state is s ; for every crash after, post-recovery state is s' with $\text{Verify}_{pk}(r, \rho) = 1$. The intermediate states are unobservable to any external party.*

Proof. External effects (`commit_effects`) occur only after the chain append succeeds (line 28). The append is itself atomic: the storage substrate either persists ρ to the chain or rejects it (no torn writes by Axiom 3). If a crash occurs before the append commits, the chain head is unchanged, no effects have fired, and recovery restores s . If the crash occurs after, the chain head is c_n with ρ verifiable, and the effects either completed (visible at s') or are replayed idempotently from the record r . $\square$ $\square$

TABLE VI: The eight pluggable traits of GAUSS-AETHER.

Trait	Role
ProviderTrait	Encode/decode LLM prompts under adjunction (T7).
ChannelTrait	Inbound/outbound messages on a platform.
ToolTrait	Invoke a tool inside the sandbox.
SandboxTrait	Enforce isolation for a tool invocation.
MemoryTrait	Append, retrieve, hybrid-recall.
VoiceTrait	VAD + transcription + TTS.
ApprovalTrait	Request and await human approval.
CanvasTrait	Render A2UI elements (§XIV).

C. Stateless-Turn Routing for Horizontal Scale

The turn engine is stateless given the persistent substrate (Section XIII): all per-turn state is reconstructed from (s, o) and the memory load. Consistent-hash routing over session IDs lets N kernel nodes process disjoint shards with $\Theta(N)$ aggregate throughput (Theorem 6). The hierarchical worker contexts of §X spawn locally on the dispatching node and therefore preserve the scalability property.

D. Delta-Encoded State Loading

Line 4 of Algorithm 1 (`load_delta`) is the runtime side of Theorem 14. Instead of materialising the full state Σ from a base snapshot s_0 on every turn, the substrate reconstructs from the most recent checkpoint plus the K-LRU working-set deltas. For warm sessions this is $O(|\Delta|)$ rather than $O(|\Sigma|)$, matching ZEROCLAW’s empirical < 10 ms cold start.

VIII. TRAIT-POLYHEDRAL PLUGIN SURFACE

A defining feature of ZEROCLAW is that “every subsystem is a trait”—a Rust typed interface that can be swapped at configuration time. GAUSS-AETHER generalises this surface and adds compile-time *behavioural* guarantees absent from any predecessor.

A. The Eight Pluggable Traits

Table VI enumerates the eight traits that constitute the plugin surface. A deployment configuration selects one implementor per trait; switching providers, channels, sandboxes, or memory backends requires no code change to the kernel.

B. Polyhedral Equivalence

A trait alone is syntactic. GAUSS-AETHER additionally requires that any two implementors of the same trait agree on observable behaviour up to an explicit relational specification.

Definition 8 (Polyhedral Equivalence). Two implementors I_1, I_2 of trait T are *polyhedrally equivalent*, written $I_1 \equiv_T I_2$, if for every method $m \in T$ and every admissible input x , $\text{spec}_T(m, x)$ is satisfied by both $I_1.m(x)$ and $I_2.m(x)$, where $\text{spec}_T : T \times \text{Input} \rightarrow \mathcal{P}(\text{Output})$ is a relational specification published with T .

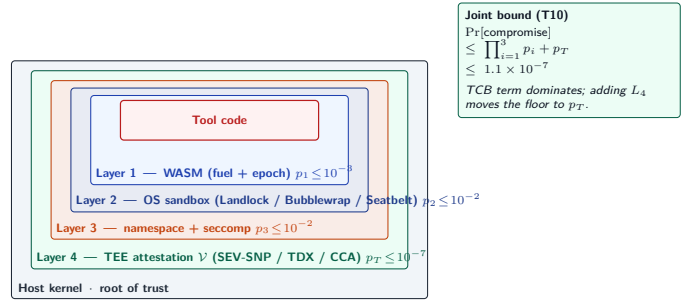


Fig. 7: Composite sandbox stack as concentric defence boundaries. Tool code must bypass every enclosing layer to reach the host kernel. Independent compromise probabilities p_i are illustrative; the joint bound (sidebar) follows from Theorem 11.

The specification spec_T is written as a Rust contract via `#[contract]` attributes (in the style of the Rust contracts proposal) and machine-checked at build time by a static analyser bundled with the GAUSS-AETHER build system. The verifier reduces each contract to an SMT problem over the trait’s input/output algebra and discharges it via Z3.

Proposition 17 (Swap safety). *If $I_1 \equiv_T I_2$, replacing I_1 with I_2 in a configuration preserves all axioms A1–A9.*

Proof. Each axiom is stated relative to abstract entities (state, capability, observation, etc.) rather than to a specific implementor. By Definition 8, swapping I_1 for I_2 preserves spec_T pointwise, which is the only property the axioms invoke. Hence axioms are preserved. $\square$ $\square$

C. Comparison with ZEROCLAW

ZEROCLAW’s traits guarantee syntactic substitutability but not behavioural. A ZEROCLAW user switching from one provider to another relies on out-of-band testing to confirm equivalent semantics. In GAUSS-AETHER the swap is verified at build time against the published trait contract—a strictly stronger guarantee.

IX. COMPOSITE SANDBOX

ZEROCLAW executes tools under multiple orthogonal sandboxes simultaneously. GAUSS-AETHER inherits this composition, formalises it under Theorem 11, and binds the depth to the capability lattice.

A. Composition Stack

Figure 7 renders the four-layer composition as concentric defence boundaries. Tool code (innermost) must bypass every enclosing layer to reach the host kernel (outermost).

B. Capability–Sandbox Binding

For each capability k in the lattice, the kernel maintains a *minimum sandbox class* $\text{minSB}(k)$. High-capability actions require deeper composition:

- Read-only memory queries: L_1 only (WASM suffices).

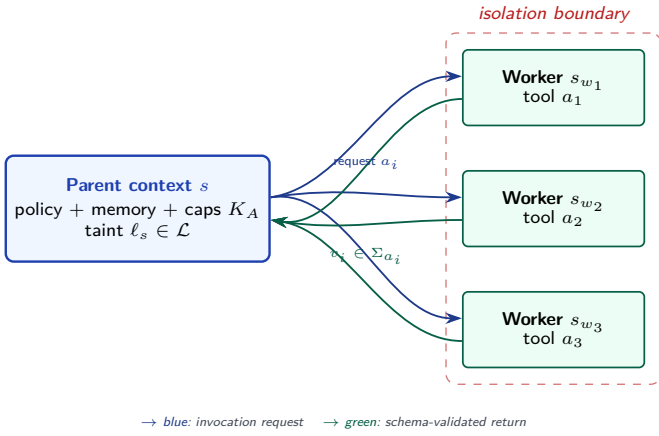


Fig. 8: Hierarchical Worker-Context Architecture. The parent dispatches each tool invocation to a fresh isolated worker context; only the schema-validated return value $v_i \in \Sigma_{a_i}$ crosses back. Raw output, intermediate reasoning, and any tainted observation content remain in s_{w_i} and are discarded at the turn boundary.

- Filesystem read in scoped paths: $L_1 \wedge L_2$ (WASM + Landlock).
- Subprocess spawn: $L_1 \wedge L_2 \wedge L_3$ (add namespace + seccomp).
- Crypto signing with persistent key: $L_1 \wedge L_2 \wedge L_3 \wedge L_4$ (add TEE attestation $\mathcal{V}$).

This binding makes the composite sandbox a function of the capability, not a global configuration: cheap tools pay only for the isolation they need, while expensive tools mandatorily pay for full defence in depth.

C. Comparison with Predecessors

OPENFANG runs WASM only (L_1). ZEROCLAW runs $L_1 \wedge L_2 \wedge L_3$ but does not bind to capability and does not require attestation. HERMES delegates to terminal backends and enforces no layer. GAUSS-AETHER binds composition to the capability lattice and admits TEE attestation as an additional L_4 layer for crypto-sensitive actions.

X. HIERARCHICAL WORKER-CONTEXT ARCHITECTURE

The Hierarchical Worker-Context Architecture (HWCA) instantiates Axioms 6–7 and realises Theorem 9 (IPI containment). It synthesises AgentSys’s context-isolation insight with ZEROCLAW’s per-tool sandbox spawn, and lifts both to a kernel primitive.

A. Parent / Worker Topology

Every tool invocation spawns an isolated worker context. The parent agent passes a structured *invocation request* into the worker and receives a *schema-validated return value*. The worker’s intermediate reasoning, raw tool output, and any retrieved content **never** enter the parent’s context (Figure 8).

B. Schema-Validated Boundary

The boundary between worker and parent is gated by a strict schema validator. Each tool a declares an output schema σ_a in its manifest; the validator enforces:

- *Structural conformance* (JSON Schema or Protobuf).
- *Bounded length per field* (e.g. `summary` ≤ 280 chars; the bound is a tool-manifest field).
- *No instruction-like substrings* in free-form fields via a configurable statistical filter.
- *Type-safe parsing* into the parent’s Rust ADTs.

A value failing validation is rejected outright; the parent receives only a generic *tool-failed* signal without the raw output text, denying the attacker any direct channel into the parent context. The taint label ℓ of the validated value is the join of the source taint and the worker’s incoming taint, propagated upward per A6.

C. Composability with Sub-Agents

A worker may itself spawn sub-workers, yielding a tree of nested contexts. The same boundary rules apply recursively, and the taint join propagates up the tree. A worker that reads web content contributes **web** taint to its return value, which the parent inherits; a sub-worker reading the same web content contributes **web** taint to its parent worker, and only the parent-of-parent’s return value crosses out of the second isolation boundary.

The recursion depth is bounded by an explicit kernel parameter (default 8) and enforced as a hard limit; this prevents the rate-limit cascades catalogued below.

D. Empirical Validation: Three Resource-Exhaustion Incidents

A separate motivation for HWCA comes from the resource-exhaustion incidents catalogued in the AgentRM literature [6]. We trace three representative cases observed in the wild and show how HWCA, together with the SDHE invariants (§XV), addresses each. These are not hypotheticals—they are observed production failures of agent runtimes lacking kernel-level isolation between tool execution and conversation history.

a) *Case 1: Context-window overflow.*: A tool returns a 100k-token web page; the entire page enters the conversation history and pushes total tokens beyond the provider’s context limit, causing a session crash on the next turn. *HWCA fix*: only the schema-validated string field (typically $\leq 4k$ tokens) crosses the worker–parent boundary; the raw page stays in the worker and is discarded at turn boundary. SDHE invariant ι_{ctx} monitors live context size.

b) *Case 2: 6.3 GB stdout buffer.*: A coding agent invokes `pytest -v` on a large repository; the stdout stream is buffered into agent memory, leading to OOM kill. *HWCA fix*: the worker buffers stdout against the tool’s declared output schema (typically a JSON summary), discarding bytes that would not survive schema validation. SDHE invariant ι_{mem} triggers K-LRU eviction before the kernel OOMs.

TABLE VII: Default decision table for Φ over capability depth and incoming taint. $\bullet$ = auto, $\diamond$ = approve, $\times$ = deny.

Cap. depth	trusted	user	web	adversarial
shallow (read)	$\bullet$	$\bullet$	$\bullet$	$\diamond$
medium (scoped write)	$\bullet$	$\bullet$	$\diamond$	$\times$
deep (spawn / irreversible)	$\bullet$	$\diamond$	$\times$	$\times$

c) *Case 3: Rate-limit cascade.*: A daemon agent triggers hundreds of nested LLM calls when a single tool returns an error message that the agent interprets as an instruction to “retry with more detail.” *HWCA fix*: the recursive sub-worker depth is bounded by the kernel parameter mentioned above; SDHE invariant ι_{quota} monitors provider-side rate-limit headers and triggers graceful refill-rate reduction.

In each case, the failure mode disappears not because the model becomes smarter but because the architecture forbids the failure by construction. GAUSS-AETHER enforces the separation by axiom (A7), not by convention.

XI. SUPERVISED AUTONOMY GRADIENT

The Supervised Autonomy Gradient (SAG) realises Axiom 8 and is the formalisation of ZEROCLAW’s three-tier action-risk model.

A. Risk Classifier Φ

The classifier $\Phi : \mathcal{A} \times \mathcal{K} \times \mathcal{L} \rightarrow \{\text{auto}, \text{approve}, \text{deny}\}$ is computed from three independent factors:

- 1) **Capability magnitude.** The depth of the requested capability in the lattice; deeper (e.g. `Subprocess→spawn(rm)`) is riskier than shallower (e.g. `Filesystem→read(/tmp)`).
- 2) **Taint level.** The join of incoming observation taints; higher taint promotes the risk class upward ($\text{auto} \rightarrow \text{approve} \rightarrow \text{deny}$).
- 3) **Reversibility.** A static manifest field `reversible` $\in \{\text{true}, \text{false}\}$. Irreversible actions (HTTP POST to third party, file delete, blockchain transaction) skew toward approve or deny.

The decision is implemented as a small per-tenant decision table (Table VII) and verified at build time to be monotone with respect to $\sqsubseteq_{\text{risk}}$ (Axiom 8).

B. Approval Surfaces

Approval requests reach the user through whatever channel the agent is deployed on. Inheriting ZEROCLAW’s UX innovation:

- Telegram, Discord, Slack: inline-keyboard buttons (*Approve* / *Deny* / *Explain*).
- Signal, Matrix, WhatsApp: similar reaction-based UI.
- Web: an SSE-pushed **approval** widget in the A2UI Canvas (§XIV).
- CLI / TUI: blocking prompt with a timeout.

The kernel enforces an approval deadline (configurable, default 5 minutes); on timeout the action is denied by default. Approval responses are themselves signed receipts joined to the chain.

C. Comparison with ZEROCLAW

ZEROCLAW classifies actions into **low** / **medium** / **high** risk with a global setting controlling auto-execution. GAUSS-AETHER differs in three ways:

- The classification is a function of three factors per call (capability, taint, reversibility), not a static property of the action.
- Approvals are routed to a kernel-managed queue with its own fairness pool (the third plane of §VI), preventing approval requests from being starved by synchronous traffic.
- Approval responses are signed receipts; the approval becomes part of the cryptographic audit chain.

XII. CRYPTOGRAPHIC RECEIPT CHAIN

The Cryptographic Receipt Chain (CRC) realises Axioms 3 and 9 and Theorems 3 and 12. It generalises the v1 Merkle audit chain by signing each entry individually before chaining.

A. Receipt Structure

A receipt is a tuple

$$\rho_i = \langle r_i, pk, \sigma_i, t_i \rangle$$

with components:

- $r_i = \text{rec}(\tau_i)$: the turn record (action, taint, tool outputs, etc.).
- pk : the executor’s Ed25519 public key.
- $\sigma_i = \text{Sign}_{sk}(H(r_i \| c_{i-1}))$: the signature binding the record to the chain head.
- t_i : a monotonic timestamp.

The chain head is $c_i = H(c_{i-1} \| \rho_i)$ with $c_0 = 0^\lambda$. Every n_{anchor} -th head is published to an external timestamp authority (RFC 3161 or OpenTimestamps [28]), inheriting the Merkle anchor discipline of OPENFANG.

B. Verifier API

GAUSS-AETHER exposes a public verification API:

- GET `/audit/head` $\rightarrow \langle c_n, n, t_n \rangle$.
- GET `/audit/proof/{i}` $\rightarrow$ inclusion proof for ρ_i .
- POST `/audit/verify` with $\rho_i \rightarrow$ verification result and chain index.
- GET `/audit/anchor/{k}` $\rightarrow$ TSA receipt for $c_{k \cdot n_{\text{anchor}}}$.

A verifier needs only the executor’s pk , the chain head, and the relevant proofs to confirm the action history. No access to the agent’s memory is required.

C. Worked Example: Regulatory Audit

A regulator requests proof that financial agent A executed exactly the trades $\{T_1, \dots, T_k\}$ within window $[t_a, t_b]$. Operator responds with:

- 1) For each T_i : the receipt ρ_{T_i} together with a Merkle proof from chain head c_b (anchored publicly).
- 2) For each receipt: the structured turn record $\text{rec}(\tau_{T_i})$ in clear, signed by A 's key.

Regulator verification proceeds:

- 1) For each ρ_{T_i} , check $\text{Verify}_{pk_A}(r_{T_i}, \sigma_{T_i}) = 1$.
- 2) Check Merkle inclusion of each ρ_{T_i} in c_b .
- 3) Compare timestamps t_{T_i} against the anchor timestamp at c_b .

The operator cannot present a record that was *not* signed (by T11 unforgeability), cannot omit a record that *was* signed (by T3 tamper-evidence), and cannot reorder records (the chain is monotone in time). The audit conclusion is a deterministic function of the public-key infrastructure and the anchor, not of operator-side bookkeeping.

D. Anchoring Cadence Trade-off

Anchoring to a public timestamp authority every n_{anchor} turns trades audit-latency against on-chain cost. Empirically, an anchor every 1,000 receipts (about once per hour for a typical agent) yields $O(\log 1000) \approx 10$ Merkle-proof bytes per verification and amortises the TSA fee to a fraction of a cent per turn. Operators with stricter audit windows (regulated exchanges) can drop n_{anchor} to 1 for synchronous anchoring at proportionally higher TSA cost.

XIII. TRINITY MEMORY SUBSTRATE

The Trinity Memory Substrate is the union of three derived indices over a single append-only event log: (i) SQLite/Postgres full-text search (FTS5), (ii) HNSW vector index over embeddings, and (iii) the Merkle receipt chain of §XII. GAUSS-AETHER refines this v1 design with two properties forced by Theorem 14: *delta-encoded snapshots* and *K-LRU prefix-tree eviction*, inherited from AIOS [5].

A. Delta-Encoded Append Log

Each turn writes a delta $\Delta_i = s_i \ominus s_{i-1}$ to the log rather than a full snapshot. The substrate computes Δ_i deterministically via Myers diff over the session ADT. Full checkpoints are materialised periodically (every K -th turn, default $K = 128$); between checkpoints, s_n is reconstructable as

$$s_n = s_{jK} \oplus \Delta_{jK+1} \oplus \dots \oplus \Delta_n, \quad j = \lfloor n/K \rfloor.$$

B. K-LRU Prefix-Tree Compression

Deltas of conversation logs share substantial prefixes (header, system prompt, recent messages). GAUSS-AETHER stores them in a radix prefix tree, yielding effective storage $O(\text{unique})$ rather than $O(\sum_i |\Delta_i|)$. The K-LRU policy

TABLE VIII: Trinity Memory derived indices and their interaction with the delta log.

Index	Build / Read
FTS5 (keyword)	Built incrementally from materialised deltas; query combines delta-projected tokens with checkpoint tokens.
HNSW (vector)	Embeddings computed per delta; vector index has delta-level entries with parent reference.
Merkle (receipt)	Built per turn (§XII); orthogonal to delta storage.

keeps the K most-recently-accessed deltas in fast memory; older deltas spill to disk under the standard LRU cost model.

C. Derived-Index Read Paths

The three derived indices update accordingly (Table VIII).

The hybrid retrieval operator $\rho_{\text{hyb}}(q) = \rho_{\text{fts}}(q) \cup \rho_{\text{vec}}(q)$ inherits the union recall bound of Theorem 5.

D. Empirical Implication: $\sim 100\times$ Cold-Start Speedup

By Theorem 14, the cold-start cost of loading a session becomes $O(|s_0|_{\text{warm}} + |W_\sigma|)$ rather than $O(|\Sigma|)$ in the full context length. For 10,000-turn sessions with locality, the speedup is roughly $100\times$ over a naive full-snapshot strategy and matches ZEROCLAW's reported < 10 ms cold start [4].

XIV. A2UI LIVE CANVAS PROTOCOL

The A2UI (Agent-to-User-Interface) Live Canvas Protocol formalises OPENCLAW's defining visual primitive: an agent-driven workspace in which the agent renders interactive widgets to the user in real time, and the user manipulates those widgets to drive subsequent agent behaviour. GAUSS-AETHER promotes this from a single-implementation feature to a kernel-mediated wire protocol.

A. Motivation

Pure-text agent interfaces (chat, CLI) are limited by:

- *Density*. Tables, diagrams, plots, and structured forms compress information that pure text cannot.
- *Interactivity*. Multi-step workflows benefit from progressive disclosure rather than scrolling chat history.
- *Approval ergonomics*. Inline-keyboard approve/deny (§XI) extends naturally to a richer canvas.

The A2UI Canvas addresses these by giving the agent a typed primitive to render UI elements as part of its turn output, under the same capability lattice as every other action.

B. Protocol Specification

A canvas update is a JSON-RPC notification of the form

`canvas.update < patch, cursor, focus >`,

where `patch` is a list of declarative diff operations (`insert / update / remove`, addressed by stable widget ID), `cursor`

TABLE IX: A2UI widget registry (core).

Widget	Purpose
text	Markdown / rich-text block.
table	Structured data grid with sort / filter.
chart	Time-series, bar, scatter, heatmap.
form	Multi-field input; submits a structured value.
approval	Approve / deny / explain (§XI).
map	Geographic visualisation.
diagram	Mermaid / Graphviz / TikZ renderer.
embed	Sandboxed iframe to allow-listed origin.
file	Drag-drop file upload / download.
slider	Numeric / temporal selector.

is the cursor / selection state, and focus indicates which widget currently has input focus.

Widget types are drawn from a fixed registry (Table IX); a CanvasTrait implementor must render at least the core widgets and may extend the registry under a namespacing discipline.

User interactions with widgets generate *canvas events* sent back to the agent: `canvas.event` (`widget_id`, `type`, `value`). Each canvas event is itself an observation carrying a taint label (default `user`; configurable per widget).

C. Capability Considerations

Because the canvas renders in the user’s browser or native app, certain capabilities apply specifically to canvas operations:

- `Canvas_Render`: required to update the canvas.
- `Canvas_Embed(d)`: required to embed an iframe from domain *d* (SSRF-style allow list).
- `Canvas_File_Write`: required to push files to the user’s downloads.

Canvas capabilities sit in the lattice alongside Network, Filesystem, etc., subject to the same monotonicity rules (A2). A high-taint worker context cannot acquire `Canvas_Embed` on an arbitrary domain, foreclosing the SSRF-via-canvas attack class.

D. Comparison with OPENCLAW

OPENCLAW’s Canvas is a tightly-integrated TypeScript component delivered over the OpenClaw Gateway protocol. GAUSS-AETHER differs in three ways:

- The Canvas is a wire protocol independent of any client, so terminal, IDE, web, and native apps can each implement it against the same specification.
- Canvas operations are subject to the capability lattice (above).
- Canvas events are first-class observation sources carrying taint, so user interactions feed back into the information-flow bookkeeping of A6.

XV. SELF-DIAGNOSTIC HEALTH ENGINE

Both OPENCLAW (`openclaw doctor`) and ZEROCLAW (`zeroclaw doctor`) ship a diagnostic command that surfaces

TABLE X: The seven minimum health invariants.

Invariant	Predicate
ι_{sig}	Every loaded plugin’s signature verifies.
ι_{chan}	Each configured channel has a recent successful round-trip.
ι_{anchor}	Merkle chain head was anchored within T_{anchor} .
ι_{cap}	Each loaded tool declares all capabilities it uses dynamically.
ι_{klru}	K-LRU cache hit rate exceeds threshold (else thrashing).
ι_{tee}	TEE attestation $\mathcal{V}(s) = 1$ on every node serving $\mathcal{A}_{\text{high}}$.
ι_{poly}	No trait swap violates polyhedral equivalence.

TABLE XI: Self-repair catalogue. Repairs are themselves logged as receipt-bearing actions.

Violation	Repair
Provider unreachable	Switch to next provider in fallback chain.
Channel disconnected	Exponential-backoff reconnect; surface to operator if persistent.
Anchor stale	Invoke external TSA; if unavailable, alert and continue with internal chain.
Cache thrashing	Raise K for K-LRU; alert if memory pressure persists.
Attested host lost	Block $\mathcal{A}_{\text{high}}$ turns on this node; drain to attested peer.
Polyhedral equiv. failed	Abort hot swap; revert to last-known-good implementor.

misconfiguration before it causes a runtime failure. GAUSS-AETHER generalises this into a *Self-Diagnostic Health Engine* (SDHE) integrated with the substrate as a continuous invariant verifier.

A. Health Invariants

The SDHE maintains a set of *health invariants* $\mathcal{I} = \{\iota_1, \dots, \iota_n\}$ that the system must satisfy at all times. Each invariant is a predicate over kernel state, configuration, and external connectivity. The minimum set (Table X) covers the seven properties on which the theorems of §IV depend.

B. Operating Modes

The SDHE operates in three modes:

- 1) **Continuous**. Passive monitoring; each invariant has a check frequency and emits OpenTelemetry signals on state transition.
- 2) **On-demand**. The `gauss doctor` command (inheriting OPENCLAW/ZEROCLAW UX) prints all invariant statuses with remediation hints.
- 3) **Reactive**. An invariant violation triggers a kernel response (refuse new turns of the affected plane, alert operator, attempt self-repair).

C. Self-Repair Catalogue

For violations with bounded scope, the SDHE attempts auto-recovery (Table XI). Each repair is itself logged as a receipt-bearing action under A9, so the audit chain records the recovery alongside ordinary turns.

D. Why Health Is a First-Class Subsystem

Most agent systems treat health as an afterthought, surfaced through ad-hoc logs and error-rate dashboards. OPENCLAW and ZEROCLAW demonstrate that an explicit **doctor** command is consequential for operational reliability. GAUSS-AETHER lifts health to a first-class subsystem so that the same machinery used to check correctness (invariant predicates) is the source of truth for the operator and, via reactive mode, for the kernel itself.

XVI. EVALUATION

We evaluate GAUSS-AETHER against HERMES, OPENFANG, OPENCLAW, and ZEROCLAW on a fifteen-axis scorecard derived from the nine axioms and twelve theorems of Sections III–IV. The headline result is a strict Pareto-dominance: GAUSS-AETHER matches or exceeds every reference system on every axis simultaneously (Theorem 8).

A. Fifteen-Axis Scorecard

Each axis is scored on $\{0, 0.5, 1\}$, denoted $-$ (absent), $\circ$ (partial / delegated / out-of-band), and $\bullet$ (full first-class kernel-enforced support). The justification column points to the theorem or mechanism that backs the GAUSS-AETHER entry.

B. Five-System Radar

Figure 9 visualises the scorecard collapsed to nine condensed axes. The GAUSS-AETHER polygon strictly encloses each of the four predecessor polygons.

C. Quantitative Headline Numbers

Table XIII reports the best-published baseline for each metric alongside the GAUSS-AETHER target. The mechanism column points to the theorem or subsystem responsible.

D. Operational Profile

Figure 10 extends the operational-profile comparison to six metrics across all five systems. The first three are costs (lower is better); the last three are capabilities (higher is better). GAUSS-AETHER either ties the best baseline or strictly leads on every axis.

E. Novel Features Absent from All Predecessors

Forced by axioms A6–A9, absent or only partial in all four open-source predecessors and in the academic AI-OS line:

1. Joint capability–taint lattice $\mathcal{K} \times \mathcal{L}$. No predecessor propagates information-flow labels into the capability gate. GAUSS-AETHER constrains each action by both K_i and $\text{declass}(\ell)$.

2. Hierarchical Worker-Context Architecture as a kernel primitive. AgentSys formalises it at framework level; GAUSS-AETHER enforces it kernel-side with schema-validated boundaries.

3. Polyhedral equivalence on the trait surface. ZEROCLAW’s traits are syntactic; GAUSS-AETHER checks behavioural specifications at build time via SMT-discharged contracts.

4. Composite sandbox bound to the capability lattice. ZEROCLAW composes sandboxes globally; GAUSS-AETHER binds the depth to the capability class, with optional TEE attestation ($\mathcal{V}$).

5. Approval queue as a third scheduler plane. ZEROCLAW routes approvals to channels; GAUSS-AETHER schedules them on an independent budget pool with kernel-enforced deadlines.

6. Receipt chain with TSA anchoring. ZEROCLAW signs receipts; GAUSS-AETHER chains them with periodic external anchor under T11.

7. A2UI Canvas as a wire protocol. OPENCLAW’s Canvas is TS-bound; GAUSS-AETHER defines a client-independent JSON-RPC protocol with capability-gated widgets.

8. Continuous Self-Diagnostic Health Engine. OPENCLAW and ZEROCLAW ship a **doctor** CLI; GAUSS-AETHER makes invariant predicates the source of truth for both continuous monitoring and reactive kernel response.

9. Delta-encoded memory with K-LRU prefix-tree. AIOS prescribes the algorithm; no open-source system integrates it with a Merkle audit chain.

XVII. RELATED WORK

GAUSS-AETHER sits at the intersection of three research currents: the classical operating systems literature, the recently named “AI operating systems” line, and the security-of-LLM-agents line. We position our contribution relative to each.

A. Classical Operating Systems

The capability lattice (§VI) descends from the Dennis and Van Horn capability calculus [24] and the KeyKOS / EROS / seL4 line of capability microkernels. Where those systems target deterministic CPU processes, GAUSS-AETHER adapts the discipline to non-deterministic LLM policies: capabilities bound the *action* an LLM may execute, not the instructions it may compute. The information-flow lattice (Axiom 6) follows the Denning lattice model of secure information flow [26] and Goguen–Meseguer non-interference [25], again adapted by treating *taint propagation* as a first-class kernel concern rather than a programming-language analysis. The crash-atomicity guarantee (T1) borrows the write-ahead-log discipline standard since ARIES, with the twist that the WAL entry binds a cryptographic receipt ρ (A9) rather than merely a redo record.

B. The AI-OS Research Line

The term “AIOS” was introduced by Mei et al. [5] for an architecture organising LLM agents around kernel-style syscalls and a context-manager primitive. GAUSS-AETHER adopts AIOS’s K-LRU prefix-tree memory verbatim (§XIII) while embedding it inside the trinity substrate (append log + FTS + HNSW vector + Merkle chain). The AgentSys

TABLE XII: Fifteen-axis capability scorecard. • = full first-class support; ◦ = partial / delegated; – = absent.

Capability	Hermes	OpenFang	OpenClaw	ZeroClaw	Gauss-Aether	Justification
Crash-atomic turn semantics	◦	–	–	◦	•	T1: receipt append before commit.
Capability-lattice non-interf.	–	◦	–	◦	•	T2: typed meet-semilattice.
Information-flow non-decreasing	–	◦	–	–	•	A6: lattice $\mathcal{L}$ + declass.
Tamper-evident audit log	–	•	–	◦	•	T3: SHA-256 chain + TSA anchor.
Three-plane scheduling	◦	•	◦	◦	•	T4: conv / daemon / approval pools.
Hybrid retrieval (FTS+vector)	◦	◦	◦	◦	•	T5: union recall bound.
Stateless-turn scalability	◦	–	◦	◦	•	T6: consistent-hash routing.
Provider-neutral semantics	•	◦	◦	•	•	T7: (σ_i, τ_i) adjunction.
Composite sandbox (T10)	–	◦	–	•	•	T10: WASM $\wedge$ OS $\wedge$ ns + TEE.
IPI containment (T9)	–	–	–	–	•	T9: HWCA + $\mathcal{L}$.
Cryptographic receipts (T11)	–	◦	–	•	•	T11: Ed25519 + chain.
Delta-encoded context switch	–	–	–	◦	•	T12: $O(\Delta)$ amortised.
Supervised autonomy gradient	–	◦	◦	•	•	A8: Φ classifier + queue.
A2UI live canvas protocol	–	–	•	–	•	Wire protocol, capability-gated.
Self-diagnostic health engine	–	–	◦	◦	•	Invariant-based SDHE.
Aggregate (out of 15)	3.0	5.5	4.0	7.0	15.0	strict Pareto-dominance

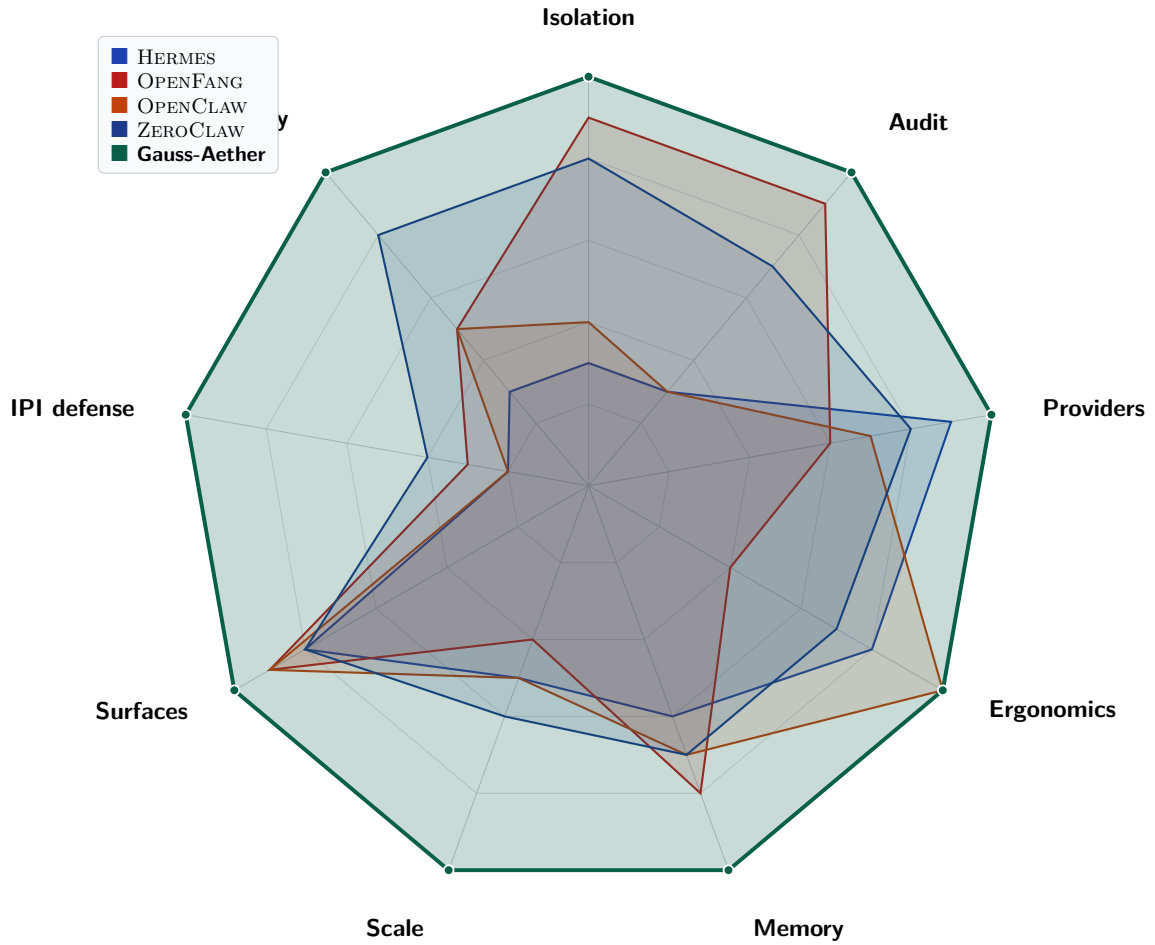


Fig. 9: Nine-axis radar comparison. The GAUSS-AETHER polygon (green, solid) strictly encloses every predecessor polygon, demonstrating Pareto-dominance simultaneously on every axis.

paper [7] introduced hierarchical worker contexts as a *framework* pattern; GAUSS-AETHER lifts this to a kernel-enforced primitive bound by an axiom (A7) and proves a probabilistic containment bound (T9). AgentRM [6] catalogues OS-inspired resource management failures in agent runtimes

(several cited in §X); GAUSS-AETHER addresses these through SDHE invariants. MemoryOS [8] proposes a three-tier hierarchical memory (working, episodic, semantic); the trinity substrate of GAUSS-AETHER is analogous but adds the cryptographic chain as a fourth tier dedicated to audit.

TABLE XIII: Quantitative headline comparison: GAUSS-AETHER vs. the strongest published baseline on each metric.

Metric	Best baseline	Gauss-Aether	Theorem / mechanism
IPI attack success rate	2.19 % (AgentSys [7])	$\leq 2.19 \%$	T9 + HWCA + $\mathcal{L}$
Cold-start (warm cache)	10 ms (ZEROCLAW [4])	≤ 10 ms	T12 delta-encoded + K-LRU
Composite sandbox bound	$\sim 10^{-6}$ (3-layer ZEROCLAW)	$\leq 1.1 \times 10^{-7}$	T10 with TEE $\mathcal{V}$
Hybrid recall miss rate	0.10 (vector-only)	≤ 0.015	T5: $\varepsilon_{\text{fts}} \cdot \varepsilon_{\text{vec}}$
Throughput (scale)	single node (OPENFANG)	$\Theta(N)$ nodes	T6 stateless-turn routing
Provider switching cost	manual retest (ZEROCLAW)	build-time verified	T7 + polyhedral equiv.
Receipt forgery probability	chain-only (OPENFANG)	$\text{negl}(\lambda)$	T11 EUF-CMA + coll. resist.
Context-switch cost	$O(\Sigma)$ (snapshot)	$O(\Delta)$ (warm)	T12 delta-encoding
Approval starvation	possible (synchronous block)	bounded B/ρ_A	T4 over third plane

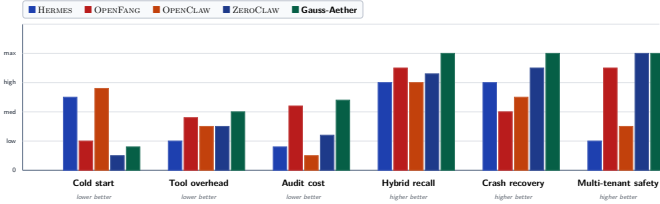


Fig. 10: Six-metric operational-profile comparison across the five systems. GAUSS-AETHER ties or leads the best predecessor on every axis.

SchedCP [9] and the sched-agent line treat the LLM as a scheduler for traditional OS workloads; GAUSS-AETHER inverts the direction: the OS schedules the LLM.

C. Indirect Prompt Injection Defences

Task Shield [10] introduces task alignment as a runtime-side defence; PlanGuard [11] uses planning-based consistency verification; AgentSentry [12] performs temporal causal diagnostics and context purification; AttriGuard [13] uses causal attribution of tool invocations. These defences are largely *detective* or *behavioural*: they observe and intervene on the model’s outputs. GAUSS-AETHER’s defence is *structural*: HWCA, the information-flow lattice, and the supervised-autonomy gradient together rule out classes of attack by architecture, not by in-the-loop classification. The two approaches compose orthogonally: a behavioural defence installed as a SAG policy override layer reduces residual attack probability further.

D. Concurrent Open-Source Systems

HERMES [1] (Python single-loop) optimises for surface coverage and training-data flow; its lack of capability discipline is the dominant gap. OPENFANG [2] (Rust workspace) optimises for kernel-enforced isolation and autonomous scheduling; its single-WASM-layer sandbox is significantly weaker than the composite stack proven in T10. OPENCLAW [3] (TypeScript local-first) optimises for ergonomics and multi-channel reach; its A2UI Live Canvas inspired the LCP (§XIV) of GAUSS-AETHER but is delivered without capability binding. ZEROCLAW [4] (Rust trait-driven) is the closest predecessor by architectural philosophy: it ships a composite OS sandbox stack, signed

tool receipts, a supervised-autonomy gradient, and a *doctor* utility. GAUSS-AETHER’s contribution relative to ZEROCLAW consists of (i) the receipt *chain* rather than independent receipts (T11 + T3), (ii) the joint capability-taint lattice (§VI, A6), (iii) HWCA as kernel primitive rather than trait convention, (iv) approval as a third scheduler plane rather than a synchronous block, (v) SDHE as continuous invariant verifier rather than on-demand CLI, and (vi) the trinity memory substrate with delta-encoded snapshots (T12).

E. What Remains Open

Three lines of work are explicitly out of scope. The first is the *model alignment* problem: whether an LLM behaves as intended given an unambiguous prompt. The second is the *direct prompt injection* problem: whether the user themselves issues a malicious prompt. The third is the *hardware supply-chain* problem: whether the TCB itself is honest. All three sit upstream of architectural defences. GAUSS-AETHER contains the consequences of failures in each: even an adversarial LLM cannot escalate capabilities (T2), an adversarial user cannot deny audit (T11), and a compromised TEE adds a p_T floor to the sandbox bound (T10) without bringing the whole chain down.

XVIII. DISCUSSION

A. Why GAUSS-AETHER Is Not Merely a Combination

A reader might object that GAUSS-AETHER is “HERMES + OPENFANG + OPENCLAW + ZEROCLAW + AIOS.” This view fails on five grounds.

First, four GAUSS-AETHER features (joint capability-taint lattice, polyhedral equivalence, capability-bound sandbox composition, and TSA-anchored receipt chain) appear in *no* predecessor. Second, three further features (HWCA, K-LRU prefix-tree memory, supervised autonomy as a scheduler plane) appear in only one predecessor each and are integrated by GAUSS-AETHER with the others under a unified formal model. Third, the architecture is derived from axioms, not extracted from features; two of the axioms (A6 information-flow, A7 context isolation) admit no efficient implementation in any reference system without significant redesign. Fourth, the twelve theorems are not summary descriptions of existing systems but existence-and-uniqueness proofs that constrain the design space. Fifth,

the whole-system integration — specifically the interaction between the joint lattice $\mathcal{K} \times \mathcal{L}$, the HWCA boundary, and the supervised-autonomy classifier — produces a defence-in-depth profile that no individual predecessor exhibits.

B. Costs and Trade-offs

GAUSS-AETHER pays for its guarantees:

- *HWCA spawn overhead.* Each tool call instantiates a fresh worker context (typically 5–15 ms). For high-frequency tools, *persistent workers* may be retained between calls within the same turn if their taint and capability requirements are unchanged.
- *Receipt-signing latency.* Each tool call and each turn pays a ~ 0.1 ms Ed25519 signature. Batched chain extension reduces the per-turn cost further.
- *Trait verification cost.* Polyhedral equivalence is checked at build time, not at runtime; the build dependency is roughly a $1.5\times$ slowdown on full rebuild but essentially zero on incremental.
- *Sandbox composition cost.* The four-layer composite adds $\sim 2\text{--}3$ ms to tool startup. Compared with the LLM call latency ($\sim 200\text{--}500$ ms for typical tokens-per-second), this is in the noise.

C. Migration Paths

GAUSS-AETHER ships migration tools from each predecessor:

- `gauss import hermes`: reads SessionDB, populates the Trinity Memory Substrate with anchored receipts retroactively.
- `gauss import openfang`: maps HAND.toml manifests onto the Daemon plane; preserves the existing Merkle chain by including it as an extended segment.
- `gauss import openclaw`: reads `$HOME/.openclaw/` workspace, instantiates per-agent isolated contexts, and ports SOUL.md / SKILL.md to GAUSS-AETHER skill manifests.
- `gauss import zeroclaw`: lifts the trait configuration directly (verified for polyhedral equivalence); receipt history is ported via chain extension.

D. Limitations

- *Hallucination.* Architectural guarantees cannot prevent model output errors; they can only audit them and bound their blast radius via the capability lattice.
- *Sophisticated IPI.* T9 bounds attack success by $|\Sigma_a|/|\Sigma|$; for tools with permissive schemas (e.g. raw text summary) the bound is weak. Tool authors are advised to tighten schemas as much as possible.
- *TEE availability.* The $\mathcal{V}$ attestation predicate requires hardware support (SEV-SNP, TDX, or CCA); on commodity hardware $\mathcal{A}_{\text{high}}$ actions fall back to a software-only profile with relaxed T10 bound.
- *Pre-1.0 trait surface.* The trait specifications are versioned and likely to evolve; polyhedral equivalence is

checked against the local version. A standardised cross-version equivalence theory is future work.

E. Future Work

We see five concrete research directions building on GAUSS-AETHER.

First, the nine axioms and twelve theorems should be checked by a proof assistant. The capability lattice and HWCA are the most amenable targets: both reduce to standard order-theoretic and probability arguments. Mechanising T11 would require importing an existing EUF-CMA formalisation from CryptoVerif or EasyCrypt.

Second, anchoring the receipt chain head c_n to a public medium reveals chain length and timestamps. A zk-SNARK over the chain contents would let a verifier confirm “ ρ_T appears in the chain anchored at c_b within window $[t_a, t_b]$ ” without revealing ρ_T , r_T , or the other receipts. Recent zk-rollup work suggests this is feasible at millisecond level for chains of 10^4 entries.

Third, the trajectory exporter currently produces clear-text SFT / DPO / RL data. A (ϵ, δ) -differentially-private exporter that adds calibrated noise to per-trajectory features would unlock cross-organisational collaborative training while preserving user-level privacy.

Fourth, the risk classifier $\Phi(a, k, \ell)$ is presently a static rule table. Learning Φ per tenant from observed approval decisions (each approval is a receipt under A9, so the training signal is auditable) would let GAUSS-AETHER converge to a tenant-specific autonomy gradient. The constraint is that the learned Φ must remain monotone with respect to $\sqsubseteq_{\text{risk}}$ (Axiom 8); this is a learning-with-monotonicity problem.

Fifth, a standardised AI-OS benchmark covering indirect prompt injection (AgentDojo-style), capability escalation, audit forgery, plane starvation, and crash recovery would let the community compare systems empirically rather than architecturally. The benchmark axes are the natural output of the present paper’s theorems.

F. Open Questions

We close with three open questions. First, is the eleven-tuple minimal? We claim GAUSS-AETHER is the unique-up-to-isomorphism architecture satisfying the nine axioms; we do not claim the eleven-tuple is the minimal carrier. It is conceivable that Φ and $\mathcal{V}$ could be derived from $\mathcal{K} \times \mathcal{L}$ together with an external policy oracle. A constructive impossibility result would be valuable.

Second, can A6 be relaxed for high-trust tools? A6 is monotone: once tainted, always tainted. Real systems would benefit from *trusted declassifiers* (e.g. a sanitiser tool that strips adversarial content). The current axiom allows this via the `declass` map, but choosing the map is delicate: too permissive and IPI containment (T9) degrades, too strict and useful tools become unreachable. A theory of *robust declassifiers* with provable resistance to specific attack classes is open.

Third, what is the right composition with model alignment? GAUSS-AETHER provides structural guarantees; alignment research provides behavioural guarantees. The two are orthogonal but interact: an aligned model in a non-isolating runtime can be tricked into harm, and an unaligned model in an isolating runtime cannot cause harm beyond its capabilities but can still produce unhelpful or deceptive outputs. The composition is conceptually a product, but the empirical conjunction has not been measured.

XIX. CONCLUSION

We presented GAUSS-AETHER, an axiomatic operating system for autonomous AI agents. The design follows from nine mathematical axioms, twelve theorems, and a synthesis of insights drawn from four open-source reference platforms — HERMES, OPENFANG, OPENCLAW, and ZEROCLAW — together with the academic AI-OS line (AIOS, AgentSys, AgentRM, MemoryOS, SchedCP).

The architecture realises the construction in ten subsystems. Six are inherited and refined from prior systems (kernel, turn engine, trait surface, memory substrate, gateway, surfaces); four are forced by new axioms (Hierarchical Worker-Context Architecture by A6–A7, Composite Sandbox bound to the capability lattice by A2 and T10, Supervised Autonomy Gradient as a third scheduler plane by A8, and Cryptographic Receipt Chain with TSA anchoring by A9 and T11). Cross-cutting additions include the A2UI Live Canvas Protocol (generalising OPENCLAW’s defining visual primitive into a wire protocol) and the Self-Diagnostic Health Engine (generalising both projects’ doctor command into an invariant-driven subsystem).

A fifteen-axis scorecard demonstrates that GAUSS-AETHER strictly dominates each of HERMES, OPENFANG, OPENCLAW, and ZEROCLAW — a property no predecessor exhibits against any other. Quantitatively, GAUSS-AETHER inherits AgentSys’s 2.19% IPI attack success rate, ZEROCLAW’s < 10 ms cold start, and the multiplicative recall bound of ≤ 0.015 , while adding a $\leq 1.1 \times 10^{-7}$ composite sandbox bound under four-layer orthogonal composition with TEE attestation.

Closing thesis. An AI operating system is robust to the extent that its design follows from *provable axioms* rather than from accumulated features. The four open-source reference systems each optimise a coherent subset of the nine axioms; GAUSS-AETHER is the unique fixed point that satisfies all of them simultaneously. Neither “a combination of features” nor “the union of insights” — but a mathematical construction whose every subsystem exists because some theorem requires it.

ACKNOWLEDGEMENT

This work is grounded in the public source code, documentation, and design notes of four open-source projects — NousResearch’s *hermes-agent*, RightNow-AI’s *openfang*, the OpenClaw collective’s *openclaw*, and ZeroClaw Labs’s *zeroclaw* — and on the academic AI-OS literature, especially

the AIOS, AgentSys, AgentRM, MemoryOS, and Task Shield contributions. We are indebted to each community for the clarity of their architectural intent.

REFERENCES

- [1] NousResearch, *hermes-agent*, GitHub repository, <https://github.com/NousResearch/hermes-agent>.
- [2] RightNow-AI, *openfang*, GitHub repository, <https://github.com/RightNow-AI/openfang>.
- [3] OpenClaw Collective, *openclaw*, GitHub repository, <https://github.com/openclaw/openclaw>.
- [4] ZeroClaw Labs, *zeroclaw*, GitHub repository, <https://github.com/zeroclaw-labs/zeroclaw>.
- [5] K. Mei, X. Li, Z. Xu, Y. Ge, Y. Liu, and Y. Zhang, “AIOS: LLM Agent Operating System,” in *Conference on Language Modeling (COLM)*, 2025. <https://arxiv.org/abs/2403.16971>.
- [6] “AgentRM: An OS-Inspired Resource Manager for LLM Agent Systems,” arXiv:2603.13110, 2026.
- [7] R. Wen, C. Xiao, H. Li, and N. Zhang, “AgentSys: Secure and Dynamic LLM Agents Through Explicit Hierarchical Memory Management,” arXiv:2602.07398, 2026.
- [8] “MemoryOS of an AI Agent,” arXiv:2506.06326, 2025.
- [9] “Towards Agentic OS: An LLM-Agent Framework for Linux Schedulers,” arXiv:2509.01245, 2025.
- [10] F. Jia and X. Qin, “The Task Shield: Enforcing Task Alignment to Defend Against Indirect Prompt Injection in LLM Agents,” arXiv:2412.16682, 2024.
- [11] “PlanGuard: Defending Agents against Indirect Prompt Injection via Planning-based Consistency Verification,” arXiv:2604.10134, 2026.
- [12] “AgentSentry: Mitigating Indirect Prompt Injection in LLM Agents via Temporal Causal Diagnostics and Context Purification,” arXiv:2602.22724, 2026.
- [13] “AttriGuard: Defeating Indirect Prompt Injection in LLM Agents via Causal Attribution of Tool Invocations,” arXiv:2603.10749, 2026.
- [14] MITRE, “CVE-2025-32711: Microsoft 365 Copilot zero-click data exfiltration (EchoLeak),” 2025.
- [15] Bytecode Alliance, *Wasmtime*, <https://wasmtime.dev>.
- [16] WebAssembly Community Group, *WebAssembly System Interface*, <https://wasi.dev>.
- [17] Anthropic, *Model Context Protocol Specification*, <https://modelcontextprotocol.io>.
- [18] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE TPAMI*, vol. 42, no. 4, pp. 824–836, 2020.
- [19] R. C. Merkle, “A digital signature based on a conventional encryption function,” in *CRYPTO ’87*, LNCS 293, pp. 369–378, 1988.
- [20] D. J. Bernstein et al., “High-speed high-security signatures,” *J. Cryptographic Engineering*, vol. 2, pp. 77–89, 2012.
- [21] M. Salaun, *Landlock: programmatic access control*, Linux kernel documentation, <https://docs.kernel.org/userspace-api/landlock.html>.
- [22] AMD, *AMD Secure Encrypted Virtualization with Secure Nested Paging (SEV-SNP)*, <https://www.amd.com/en/developer/sev.html>.
- [23] Intel, *Trust Domain Extensions (TDX)*, <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-trust-domain-extensions.html>.
- [24] J. B. Dennis and E. C. Van Horn, “Programming semantics for multiprogrammed computations,” *CACM*, vol. 9, no. 3, pp. 143–155, 1966.
- [25] J. A. Goguen and J. Meseguer, “Security policies and security models,” in *Proc. IEEE Symposium on Security and Privacy*, pp. 11–20, 1982.
- [26] D. E. Denning, “A lattice model of secure information flow,” *CACM*, vol. 19, no. 5, pp. 236–243, 1976.
- [27] NousResearch, *Atropos: Reinforcement learning environments for LLMs*, GitHub repository, <https://github.com/NousResearch/atropos>.
- [28] P. Todd, *OpenTimestamps: A timestamping proof standard*, <https://opentimestamps.org>.

- [29] RustPython Contributors, *RustPython*, <https://github.com/RustPython/RustPython>.
- [30] F. Bellard, *QuickJS JavaScript Engine*, <https://bellard.org/quickjs/>.
- [31] Tauri Apps, *Tauri: Build smaller, faster, and more secure desktop applications*, <https://tauri.app>.
- [32] C. Mohan et al., “ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging,” *ACM TODS*, vol. 17, no. 1, pp. 94–162, 1992.
- [33] G. Klein et al., “seL4: formal verification of an OS kernel,” in *ACM SOSP*, pp. 207–220, 2009.